

Stichwortverzeichnis

A

Accuracy [9.8.1](#) · (S. 8a)
Aktivierungsfunktion [10.1.2](#) · (S. 8a)
Array [7.1.1](#) · (S. 6b)
Asymptotische Notationen [5.1](#) · (S. 4b)
Ausrollen [5.7](#) · (S. 5a)

B

Balanced Accuracy [9.8.1](#) · (S. 8a)
Baumtraversierung [6.4.2](#) · (S. 5b)
Binäre Suche [4.5](#) · (S. 4b)
Boolean Mask [7.2.1](#) · (S. 7a)
Bottom-up [3.1](#) · (S. 2b)
Broadcasting [7.2.2](#) · (S. 7a)
BST / Binary Search Tree [6.4](#) · (S. 5b)
Bubble Sort [4.2.1](#) · (S. 4a)

C

Centroid [11.1.2](#) · (S. 8b)
Clustering [11.1](#) · (S. 8b)
CNN / Convolutional Neural Network [10.3](#) · (S. 8b)
Confusion Matrix [9.8.1](#) · (S. 8a)
Convolution [10.3.1](#) · (S. 8b)
Cross-Validation [9.6](#) · (S. 7b)

D

DataFrame [8.1](#) · (S. 7a)
Decision Tree [9.7.3](#) · (S. 7b)
Dictionary [1.4](#) · (S. 2a)
Dimensionsreduktion [11.2](#) · (S. 8b)
Divide and Conquer [4.3](#) · (S. 4a)
DP / Dynamische Programmierung [3.1](#) · (S. 2b)
Dunder-Methode [2.3](#) · (S. 2b)

E

Elbow-Kriterium [11.1.4](#) · (S. 8b)
Encoding [9.9](#) · (S. 8a)
enumerate [1.1.3](#) · (S. 1b)

F

Fehlender Wert [8.2.2](#) · (S. 7a)
Fibonacci [3.4.1](#) · (S. 3a)
FIFO *siehe* Queue [6.2](#)

G

Gini-Index [9.8.3](#) · (S. 8a)
Grid Search [9.6](#) · (S. 7b)
Grid-DP [3.7](#) · (S. 3b)

H

Hash-Tabelle [6.6](#) · (S. 6b)
Hauptkomponenten [11.2.2](#) · (S. 8b)
Heap [6.5](#) · (S. 6a)
Heap Sort [4.4.1](#) · (S. 4b)
Heapify [6.5.3](#) · (S. 6a)
Hyperparameter [9.7.2](#) · (S. 7b)

I

iloc [8.1.2](#) · (S. 7a)
Insertion Sort [4.2.3](#) · (S. 4a)
Intra-Cluster-Varianz [11.1.1](#) · (S. 8b)

K

K-Means [11.1.1](#) · (S. 8b)
k-means++ [11.1.3](#) · (S. 8b)
Klassifikation [9.1](#) · (S. 7a)
KNN / K-Nearest Neighbors [9.7.3](#) · (S. 7b)
Kollisionsbehandlung [6.6.3](#) · (S. 6b)

L

L'Hopital [5.3](#) · (S. 5a)
Landau-Notation *siehe* Asymptotische Notationen [5.1](#)
LCS / Longest Common Subsequence [3.6.1](#) · (S. 3b)
LIFO *siehe* Stack [6.2](#)
Lineare Regression [9.7.4](#) · (S. 8a)
Linked List [6.3](#) · (S. 5b)
List Comprehension [1.5.1](#) · (S. 2a)
Liste [1.2](#) · (S. 1b)
Lloyd-Algorithmus [11.1.3](#) · (S. 8b)
loc [8.1.2](#) · (S. 7a)
Logarithmusgesetze [5.3](#) · (S. 5a)
Logistische Regression [9.7.3](#) · (S. 7b)

M

Master-Theorem [5.7](#) · (S. 5a)
Max-Heap *siehe* Heap [6.5](#)
Mean Encoding [9.9](#) · (S. 8a)
Memoization [3.1](#) · (S. 2b)
Merge Sort [4.3.1](#) · (S. 4a)
Min-Heap *siehe* Heap [6.5](#)
ML / Machine Learning [9.1](#) · (S. 7a)
MLP / Multilayer Perceptron [10.1](#) · (S. 8a)
MSE / Mean Squared Error [9.8.4](#) · (S. 8a)

N

NaN *siehe* Fehlender Wert [8.2.2](#)
Neuronales Netzwerk [10.1](#) · (S. 8a)
NumPy [7.1](#) · (S. 6b)

O

O-Notation [5.1](#) · (S. 4b)
One-Hot-Encoding [9.9](#) · (S. 8a)
OOP / Objektorientierte Programmierung [2.1](#) · (S. 2a)
Operatorüberladung [2.3](#) · (S. 2b)
Ordinal Encoding [9.9](#) · (S. 8a)
Overfitting [9.5](#) · (S. 7b)

P

Pandas [8.1](#) · (S. 7a)
PCA / Principal Component Analysis [11.2.2](#) · (S. 8b)
Pooling [10.3.1](#) · (S. 8b)

Q

Quadtree [6.7](#) · (S. 6b)
Queue [6.2](#) · (S. 5b)
Quick Sort [4.3.2](#) · (S. 4b)

R

R2-Score [9.8.4](#) · (S. 8a)
Random Forest [9.2](#), [9.7.3](#)
Range Query [6.7.3](#) · (S. 6b)
Regression [9.7.1](#) · (S. 7b)
Rekonstruktion [3.3](#), [3.6.1](#)
Rekurrenz [3.4.1](#), [5.7](#)
ReLU [10.1.2](#) · (S. 8a)
reshape [7.2.2](#) · (S. 7a)

S

Schleifenanalyse [5.5](#) · (S. 5a)

Schleifenmuster [5.6](#) · (S. 5a)
Selection Sort [4.2.2](#) · (S. 4a)
Set [1.4](#) · (S. 2a)
Sigmoid [10.1.2](#) · (S. 8a)
Slicing [1.3.3](#) · (S. 1b)
Softmax [10.2](#) · (S. 8b)
Sortieren [1.3.5](#), [4.1](#)
Stack [6.2](#) · (S. 5b)
String [1.7.1](#) · (S. 2a)
Summen [5.4](#) · (S. 5a)
Supervised Learning *siehe* Überwachtes Lernen [9.7.1](#)

T

t-SNE [11.2.3](#) · (S. 8b)
Theta-Notation [5.1](#) · (S. 4b)
Top-down *siehe* Memoization [3.1](#)
Tupel [1.2](#) · (S. 1b)

U

Überwachtes Lernen [9.7.1](#) · (S. 7b)
Underfitting [9.5](#) · (S. 7b)
Unüberwachtes Lernen [11.1](#) · (S. 8b)

V

Vererbung [2.2](#) · (S. 2a)
Verlustfunktion [11.1.2](#) · (S. 8b)

W

Wachstumsordnung [5.2](#) · (S. 4b)

Z

zip [1.6.1](#) · (S. 2a)
Zwei-Index-DP [3.6](#) · (S. 3b)

1. Python-Syntax & Container

Inhaltsübersicht

1.1 Grundlagen & Schleifen-Idiome

1.1.1 Datentypen & Typprüfung

1.1.2 Operatoren & Logik

1.1.3 Schleifen-Kontrolle

1.1.4 Funktionen & Standardwerte

1.2 Container-Überblick

1.2.1 Container-Typen im Vergleich

1.3 Sequenzen: Listen & Tupel

1.3.1 Grundoperationen für Sequenzen

1.3.2 Veränderbarkeit & Unpacking

1.3.3 Slicing-Tricks

1.3.4 Sequenz-Operationen

1.3.5 Listen-Manipulation

1.3.6 Sortieren & Extremwerte

1.5.1 Comprehensions: List, Dictionary, Set

1.3.7 Grids: Initialisierung & Kopieren

1.4 Assoziative Container (Sets & Dictionaries)

1.4.1 Set und Dictionary im Vergleich

1.4.2 Mengen-Operationen (Sets)

1.4.3 Dictionary-Zugriff & Updates

1.4.4 Dictionary-Iteration

1.5 Container erzeugen & kombinieren

1.5.1 Comprehensions: List, Dictionary, Set

1.6 Gemeinsame Iteration mit zip

1.6.1 Element-Paarung

1.7 Strings

1.7.1 String-Manipulation

```
1.1. Grundlagen & Schleifen-Idiome
1.1.1. Datentypen & Typprüfung
zahl = 5 #int
komma = 3.14 #float
text = "hello" #string
logisch = True #boolean
leerr = None #Nullwert / leerer Wert (Prüfung mit: x is None)
leerr is None #True (Identitäts-Vergleich; bevorzugen fuer None)
#alle False (leere Container / Nullwerte)
bool(0), bool(""), bool([], bool(None)
instance(zahl, int) #True (prüft auch Unterklassen; bevorzugen!)
#Typ-Konvertierung (zu String, Integer, Float)
str(5), int("5"), float("3.14")
String-Methoden (→ 1.7.1)
```

```
1.1.2. Operatoren & Logik
# 2.5 (Float-Div), 2 (Ganzzahl-Div), 1 (Modulo)
5 / 2, 5 // 2, 5 % 2
# 8 (Potenz), 5 (Absolutwert), 3.14 (Runden)
2 ** 3, abs(-5), round(3.141, 2)
# quotient=3, rest=2 (Ganzzahl-Division und Rest zusammen)
quotient, rest = divmod(17, 5)
chr(65), ord("A") #Zahl zu Zeichen, Zeichen zu Zahl
a and b, a or b, not a #Logische Operatoren (nicht &&, ||, !)
1 < x < 10 #Verketteter Vergleich (True, wenn x zwischen 1 und 10)
x == y, x is y #Wertgleichheit vs. Speicher-Identität
```

```
1.1.3. Schleifen-Kontrolle
Iteration (for, enumerate, while)
# range(start, stop, step): stop exklusiv!
for i in range(0, 10, 2): # 0, 2, 4, 6, 8 (Schrittweite 2)
    print(i)

for i in range(5, 1, -1): # 5, 4, 3, 2 (rückwärts)
    print(i)

for zeichen in "abc": # über jedes Zeichen einzeln iterieren
    print(zeichen)

# enumerate() liefert Paare (Index, Element) → idx = Index, wert = Element
for idx, wert in enumerate(["a", "b"]):
    print(idx, wert)

# while wiederholt, solange eine beliebige Bedingung True ist
while N > 0:
    ziffer = N % 10 # letzte Ziffer holen
    N //= 10 # letzte Ziffer entfernen

# items() liefert Paare (Schlüssel, Wert) → name = Key, alter = Value
for name, alter in {"Alice": 20, "Bob": 22}.items():
    print(name, alter)
Dictionary-Iteration (→ 1.4.4) zip (→ 1.6)
```

```
Schleifensteuerung (break vs. continue)
# continue: Durchlauf überspringen | break: Schleife beenden
for i in range(1, 6):
    if i == 2:
        continue # i=2 überspringen (Rest im Durchlauf auslassen)
    if i == 4:
        break # Schleife bei i=4 sofort verlassen
    print(i) # Ausgabe: 1, 3

break und continue wirken immer NUR auf die am tiefsten verschachtelte
(innerste) Schleife (for, while), in der sie sich direkt befinden. Die äussere Schleife
läuft danach ganz normal weiter!
```

```
1.1.4. Funktionen & Standardwerte
# Funktionsdefinition
def summe(x, y):
    return x + y

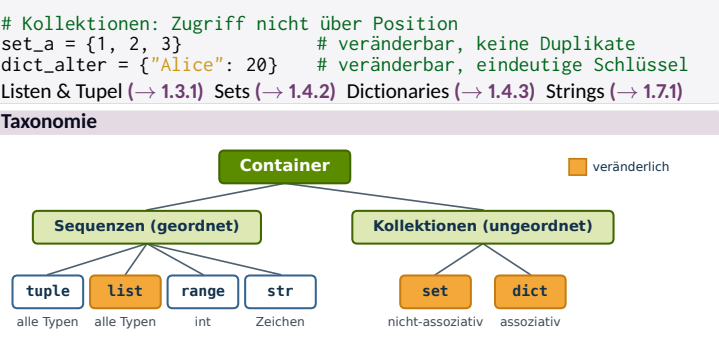
# Aufruf mit Argumenten
summe(2, 3)

def gruss(name="Gast"): # Standardwert (wird genutzt, wenn kein Wert übergeben wird)
    return "Hallo " + name

Key-Sortierung (→ 1.3.6)
```

```
1.2. Container-Überblick
1.2.1. Container-Typen im Vergleich
# Sequenzen: geordnet, Zugriff über Index
lst = [1, 2, 2] # veränderbar, Duplikate erlaubt
tup = (1, 2, 2) # unveränderbar, Duplikate erlaubt
zahlen = range(1, 4) # unveränderbare Folge von Ganzzahlen
text = "Hallo" # unveränderbare Folge von Zeichen

# Kollektionen: Zugriff nicht über Position
set_a = {1, 2, 3} # veränderbar, keine Duplikate
dict_alter = {"Alice": 20} # veränderbar, eindeutige Schlüssel
Listen & Tupel (→ 1.3.1) Sets (→ 1.4.2) Dictionaries (→ 1.4.3) Strings (→ 1.7.1)
```



```
1.3. Sequenzen: Listen & Tupel
1.3.1. Grundoperationen für Sequenzen
Elemente suchen & prüfen
len(lst) # gibt die Anzahl der Elemente
wert in daten # prüft direkt, ob der Wert enthalten ist
wert not in daten # prüft direkt, ob der Wert fehlt
daten.count(wert) # zählt den Wert ohne eigene Schleife
daten.index(wert) # Position des ersten Treffers

# True, wenn die Bedingung für mindestens ein Element gilt
any([bedingung(element) for element in daten])
# True, wenn die Bedingung für alle Elemente gilt
all([bedingung(element) for element in daten])
Listen-Methoden (→ 1.3.5) Comprehensions (→ 1.5.1)
```

```
Zugriff, Mathe & Sortierung
lst[0], tup[0] # Indezzugriff
lst[1:3] # Slicing
lst[start:stop:step] # Slicing: Ausschnitt holen (start inkl., stop exkl.)
max(lst, default=0), min(lst) # Maximum / Minimum (default sichert gegen leere Liste ab)
sum(lst) # Summe aller Elemente
list("abc") # erzeugt die Liste ['a', 'b', 'c']
tuple(lst) # in tuple umwandeln
sorted(lst) # erzeugt neue sortierte Liste (Original bleibt gleich)
Slicing (→ 1.3.3)
```

```
1.3.2. Veränderbarkeit & Unpacking
# veränderbare Sequenz
lst = [10, 20, 30]
# unveränderbare Sequenz
tup = (10, 20, 30)
# unveränderbare Zeichenkette
s = "abc"
lst[0], tup[1], s[2] # Indezzugriff
lst[1:] # Slicing

# Sequenz-Unpacking
tup_3 = (1, 2, 3)
a, b, c = tup_3 # a=1, b=2, c=3 (Entpacken)
a, b = b, a # Wertetausch ohne Hilfsvariable

# Unpacking mit * (Rest-Elemente sammeln)
first, *rest = [1, 2, 3, 4] # first=1, rest=[2, 3, 4]
*init, last = [1, 2, 3, 4] # init=[1, 2, 3], last=4
Slicing (→ 1.3.3)
```

```
1.3.3. Slicing-Tricks
lst = [10, 20, 30, 40, 50] # lst[start:stop:step]
# start inklusive, stop exklusiv, Standardschritt=1
lst[1:4] # [20, 30, 40] (von Index 1 bis 3) | lst[:]=Kopie
lst[-1] # 50 (letztes Element) | lst[-2:] = die letzten beiden
lst[::-2] # [10, 30, 50] | lst[::-1] = umgedrehte Kopie
lst[4:1:-1] # [50, 40, 30] (rückwärts von Index 4 bis 2)
kopie = lst[:] # Kopie der Liste (Änderung an kopie ändert lst nicht)
lst[1:3] = [0, 0, 0] # ersetzt Bereich durch 3 Werte (Liste wird länger!)
lst[1:3] = [] # löscht den Bereich aus der Liste
list(reversed(lst)) # umgedrehte Kopie; reversed(lst) zum Rückwärts-Iterieren
"python"[::-1] # erzeugt umgedrehten neuen String
String-Methoden (→ 1.7.1)
```

```
1.3.4. Sequenz-Operationen
[1, 2] + [3, 4] # Verkettung: verbindet zu [1, 2, 3, 4]
[1] * 3 # Wiederholung: erzeugt [1, 1, 1]
# Listen automatisch aus Variablen erzeugen
lst = [] # leere Liste zum späteren Befüllen
lst = [startwert] * n # erzeugt n gleiche Startwerte
lst = list(range(n)) # erzeugt Werte von 0 bis n-1 ([0, 1, ..., n-1])
lst = list(range(1, 10, 2)) # [1, 3, 5, 7, 9] (start, stop, step)
lst = [1, 2, 3]
# Kopieren vs. Aliasing
b = lst # Alias: Änderungen an b betreffen auch lst!
b = lst[:] # flache Kopie (Änderung an b betrifft lst nicht)

# ueber Sequenz iterieren
for element in lst:
    print(element)

Aus vorhandenen Daten erzeugen oder filtern: Comprehensions (→ 1.5.1)
```

```
1.3.5. Listen-Manipulation
lst.append(4) # ans Ende anfügen
lst.extend([5, 6]) # fügt Elemente einer Liste einzeln hinten an
# fügt 99 an Index 1 ein (verschiebt den Rest nach hinten)
lst.insert(1, 99)
# sucht und löscht die erste 1 (Fehler falls nicht da)
lst.remove(1)
# löscht und gibt letztes Element zurück (pop(i) löscht an Index i)
lst.pop()
del lst[0] #per Index löschen
kopie = lst.copy() # flache Kopie der Liste
lst.clear() # alle Elemente löschen
lst.sort() # sortiert direkt (Achtung: gibt None zurück!) (→ 4)
lst.reverse() # dreht Reihenfolge direkt um (ändert Original)
```

```
1.3.6. Sortieren & Extremwerte
# sorted() erzeugt neue sortierte Liste, Original bleibt gleich
sorted([-3, 1, -2], key=abs) # [1, -2, -3] (nach Absolutwert)
sorted(lst_namen, key=len) # sortiert nach Wortlaenge
sorted(lst_namen, key=lambda x: (len(x), x)) # Tupel-Sort: erst
Laenge, dann alphabetisch (lexikographisch)
# max() / min() unterstuetzen auch key= und default=
max(lst_namen, key=len, default="") # laengstes Wort (default
sichert leere Liste ab)

1.3.7. Grids: Initialisierung & Kopieren
2D-Grid anlegen & Deepcopy
# 2D-Grid richtig initialisieren (vermeidet Referenz-Sharing)
grid = [[0] * n for _ in range(m)] # m unabhaengige Zeilen
# [[0]*n]*m erzeugt nur m Referenzen auf dieselbe Zeile!
Comprehensions (→ 1.5.1)

1.4. Assoziative Container (Sets & Dictionaries)
1.4.1. Set und Dictionary im Vergleich
# set: ungeordnet, keine Duplikate. Schnelle Suche mit 'in'
set_a = {1, 2, 3}
set_a = set() # leeres Set; {} waere ein leeres Dict
set_a = set(lst) # aus Daten; entfernt Duplikate

alter = {"Alice": 20, "Bob": 22} # dict: Schlüssel-Wert
# sets sind schnell für Enthaltensein-Prüfung,
# kein Indexzugriff
2 in set_a # Enthaltensein prüfen
len(set_a) # Anzahl Elemente
Automatisch erzeugen oder filtern: Comprehensions (→ 1.5.1)

1.4.2. Mengen-Operationen (Sets)
set_a = {1, 2, 3}, set_b = {3, 4, 5}
set_a | set_b # Vereinigung: alle Elemente aus Set A und Set B
set_a & set_b # Schnittmenge: nur Elemente, die in beiden sind
# Differenz: Elemente in Set A, die nicht in Set B sind
set_a - set_b
set_a ^ set_b # Elemente, die genau in einem Set sind
set_a.add(6) # Element hinzufügen
set_a.remove(1) # löscht 1 aus Set (Fehler falls nicht da)
set_a.discard(1) # löscht 1 sicher; kein Fehler falls nicht da
set_a.update([6, 7]) # fügt alle Elemente aus den Daten hinzu
# True, wenn beide Sets keine gemeinsamen Elemente haben
set_a.isdisjoint(set_b)
x in set_a # True, wenn x in Set A enthalten

1.4.3. Dictionary-Zugriff & Updates
daten = dict() # leeres Dictionary zum schrittweisen Befüllen
daten = dict(paare) # erzeugt Dictionary aus Schlüssel-Wert-Paaren
# verbindet zwei Container zu Schlüssel und Wert
daten = dict(zip(keys, werte))
# alle Namen als Schlüssel mit Startwert 0
daten = dict.fromkeys(namen, 0)
Automatisch aus Daten erzeugen: Comprehensions (→ 1.5.1)
alter = {Älice: 20, "Bob": 22}
alter["Charlie"] = 25 # Schlüssel hinzufügen / aktualisieren
alter.get("Alice", 0) # holt Wert für Älice", sonst 0
# fehlender Schlüssel startet bei 0, vorhandener Wert wird erhöht
daten[key] = daten.get(key, 0) + wert
# löscht sicher: gibt None zurück, falls Key fehlt (kein Fehler)
alter.pop("Bob", None)
del alter["Bob"] # löscht Schlüssel "Bob" (Fehler falls nicht da)
"Alice" in alter # Enthaltensein prüfen
list(alter.keys()) # Liste der Schlüssel (Namen)
list(alter.values()) # Liste der Werte (Alter)
list(alter.items()) # Liste der (Schlüssel, Wert)-Paare
# liefert Schluessel mit groesstem Wert (Charlie)
max(alter, key=alter.get)
{**alter, **neu} # erzeugt zusammengefuegte Kopie (Mergen)
```

```
1.4.4. Dictionary-Iteration
alter = {"Alice": 20, "Bob": 22}

for name in alter: # Achtung: Iteriert standardmässig
    NUR über Schlüssel!
    print(name)

for jahre in alter.values(): # Werte
    print(jahre)

for name, jahre in alter.items(): # Schlüssel und Werte
    print(name, jahre)

# mit Index
for idx, (name, jahre) in enumerate(alter.items()):
    print(idx, name, jahre)
Schleifen & enumerate (→ 1.1.3) zip (→ 1.6)

1.5. Container erzeugen & kombinieren
1.5.1. Comprehensions: List, Dictionary, Set
# Form: Ausdruck für Element in Daten, optional mit if Bedingung
namen = ["Ada", "Bob", "Ada Lovelace"]

# List: Elemente verarbeiten oder filtern
[name.upper() for name in namen] # neue Liste, Namen gross
[name for name in namen if len(name) > 3] # nur passende Namen
# Ternary Operator: if-else steht vor dem for
[zahl if zahl > 0 else 0 for zahl in [-1, 2, -3]]
# verschachtelte Schleifen: links steht die aeußere, rechts die
innere
[(aussen, innen) for aussen in [1,2] for innen in [3,4]]

# Dict: Schluessel und Werte aus vorhandenen Daten erzeugen
{name: 0 for name in namen} # gleicher Startwert je Name
{name: len(name) for name in namen} # Wert aus Element berechnen

# Set: verarbeiten; gleiche Ergebnisse nur einmal speichern
{name.lower() for name in namen} # erzeugt Set ohne Duplikate

# Container mit * zusammensetzen
kombiniert = [*liste_a, *liste_b] # verbindet zwei Listen
alle_werte = {*set_a, *set_b} # verbindet Sets, Duplikate
nur einmal

Listen-Methoden (→ 1.3.5) Sets (→ 1.4.2) Dictionaries (→ 1.4.3)

1.6. Gemeinsame Iteration mit zip
1.6.1. Element-Paarung
namen = ["Alice", "Bob", "Charlie"]
alter = [20, 22, 25]
paare = list(zip(namen, alter)) # gleiche Positionen zu Paaren
verbinden
print(paare) # [("Alice", 20), ("Bob", 22), ("Charlie", 25)]
for name, jahre in zip(namen, alter): # auch in for-Schleifen
    verwendbar
    print(name, jahre)

# Gemeinsame Werte: Reihenfolge von namen bleibt erhalten
andere_namen = ["Bob", "Charlie", "Dana"]
gemeinsam = [name for name in namen if name in andere_namen]
# ["Bob", "Charlie"]

daten = ["A", "B", "C"]
# Nachbarn vergleichen: jedes Element mit seinem direkten
Nachfolger
for vorher, aktuell in zip(daten, daten[1:]):
    print(vorher, aktuell) # ("A", "B"), dann ("B", "C")

# Drei aufeinanderfolgende Werte gleichzeitig prüfen
for links, mitte, rechts in zip(daten, daten[1:], daten[2:]):
    print(links, mitte, rechts)
Dictionary aus Paaren (→ 1.4.3)
```

```
1.7. Strings
1.7.1. String-Manipulation
text = " Hello World "
text.strip() # entfernt Leerzeichen (auch \n, \t) an
Enden | strip("!?") entfernt custom Zeichen an Enden
text.lower(), text.upper() # alles klein / gross schreiben
text.startswith("He") # True (prueft Praefix)
text.endswith("ld") # True (prueft Suffix)
text.find("lo") # Index des 1. Treffers (von links), -1
falls nicht da
text.rfind("l") # Index des 1. Treffers von rechts
(Zeitspar-Abkuerzung!), -1 falls nicht da
text.count("l") # zaehlt Vorkommen des Teilstrings
text.replace("World", "Python") # ersetzt ALLE Vorkommen im
gesamten String (replace("a", "") loescht)

# Aufteilen und Verbinden
"Hello" + " " + "World" # "Hello World" (zusammenfuegen /
verketten mit +)
text.split() # trennt an Leerzeichen (entfernt auch
doppelte)
text.split("o") # trennt an custom Zeichen
"".join(["a", "b"]) # verbindet Liste zu String → "a-b"
"".join(list("abc")) # baut String aus Zeichenliste

# Pruefung und Umwandlung
"123".isdigit() # True (nur Ziffern?) | isalpha() (nur
Buchstaben?) | isalnum() (beides?) | islower()/isupper()
"lo" in "hello" # True (Teilstring-Suche /
Enthaltensein)
list("abc") # ['a', 'b', 'c'] (in Liste umwandeln)
Slicing (→ 1.3.3)

2. Klassen & OOP
2.1. Klasse definieren
Klasse definieren und verwenden
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    def honk(self):
        return f"{self.model} is honking!"

    def announce(self):
        return self.honk() # eigene Methode aufrufen

car = Car("Volvo", "EX30") # car wird in __init__ zu self
message = car.honk() # car wird automatisch als self übergeben
model = car.model # Attribut des Objekts lesen

self und Methodenaufruf
• Methode definieren — def method(self, argument) — self bezeichnet das
aktuelle Objekt.
• Von aussen aufrufen — object.method(argument) — das Objekt wird auto-
matisch als self übergeben.
• Innerhalb aufrufen — self.method(argument) — Methode desselben Ob-
jekts verwenden.
• Zustand speichern — self.attribute bleibt im Objekt gespeichert; eine
lokale Variable nur während des Aufrufs.

2.2. Vererbung
Oberklasse / Unterklasse
Eine Oberklasse (Superclass) ist eine Klasse, die als Basis für eine andere Klasse
dient – die sogenannte Unterklasse (Subclass). Mittels super().__init__() kann
die Unterklasse auf den Konstruktor der Oberklasse zugreifen.
```

```
super() in der Unterklasse
class ElectricCar(Car): # Erbt alle Attribute & Methoden von Car
    def __init__(self, brand, model):
        super().__init__(brand, model) # Car.__init__ erzeugt
            self.brand & self.model

    def charge(self):
        return f"[self.model] battery is charging." # Greift auf
            self.model aus Car zu
```

2.3. Dunder-Methoden & Operator-Überladung

```
Rückgabewert
__add__ bestimmt das Ergebnis von objekt_a + objekt_b. Der Rückgabewert
muss zur Bedeutung des Operators passen: Häufig ist er eine neue Instanz
derselben Klasse, andere Ergebnistypen sind aber ebenfalls möglich.

__add__ (Beispiel: Reissverschluss-Logik)
class CustomString:
    def __init__(self, string):
        self.string = string

    def __str__(self): # Bestimmt Textausgabe bei print(objekt)
        return self.string

    def __add__(self, other): # self = links (s1), other = rechts
        (s2)
        result_string = ""
        for i in range(len(self.string)):
            if i < len(other.string):
                result_string += (self.string[i]
                    + other.string[i])

            else:
                result_string += self.string[i]
        return CustomString(result_string) # Gibt neues Objekt
            zurück

s1 = CustomString("abcd")
s2 = CustomString("123") # s1 + s2 ruft s1.__add__(s2) auf
print(s1 + s2) # ->a1b2c3d
```

3. Dynamische Programmierung

DP klingt komplizierter als es ist: Am Ende des Tages ist es nur eine optimierte Form der Rekursion mit Speicher.

3.1. Theorie und Implementierungsarten

Kernidee & Voraussetzungen

Problem in kleinere Teilprobleme zerlegen, nur einmal lösen und Ergebnisse zur Wiederverwendung speichern.

- **Überlappende Teilprobleme** — Dieselben Teilprobleme treten mehrfach auf (Speichern vermeidet Doppelberechnung).
- **Optimale Teilstruktur** — Gesamtlösung lässt sich aus optimalen Lösungen der Teilprobleme zusammensetzen.

Top-down und Bottom-up

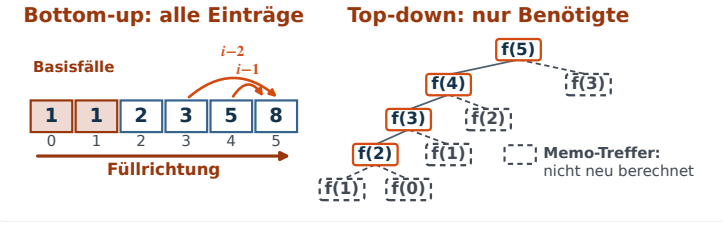
Ansatz	Berechnung	Eigenschaft
Memoization	rekursiv vom Zielzustand	nur benötigte Zustände
Bottom-up	iterativ ab Basisfällen	Zustände systematisch füllen

Theoretische Sicht

Die Zustände hängen voneinander ab (wie ein Stammbaum): Top-down sucht die Lösung rekursiv von oben nach unten (bis zur Basis); Bottom-up füllt die Tabelle mit Schleifen von unten nach oben.

- **Zeit** — Anzahl berechneter Zustände × Aufwand pro Zustand.
- **Speicher** — Anzahl gespeicherter Zustände; bei Top-down + Rekursionsstack.
- **Korrektheit** — Basisfälle korrekt ⇒ alle Folgezustände korrekt (Induktion über DAG).

Top-down und Bottom-up: Füllprinzip



3.2. Eine DP als Top-down oder Bottom-up

3.2.1. Zustand und Speicher festlegen

Funktionsparameter bestimmen den Zustand

Rekursion	Memo-Schlüssel	Bottom-up-Speicher
solve(i)	i	dp[i]: Liste
solve(i, j)	(i, j)	dp[i][j]: Tabelle
solve(i, rest)	(i, rest)	dp[i][rest]: Tabelle
unregelmässige Zustände	unveränderliches Tupel	Dictionary mit demselben Schlüssel

Was gehört zum Zustand?

Alle Parameter, die sich zwischen Aufrufen ändern und das Ergebnis beeinflussen. Unveränderte Eingaben wie grid oder text bleiben normale Argumente. **Test:** Gleicher Memo-Schlüssel muss immer dasselbe Teilproblem und Ergebnis bedeuten; sonst fehlt ein Parameter im Zustand.

3.2.2. Basisfälle und ungültige Zustände

Bekannte und unmögliche Zustände behandeln

Bedeutung	Top-down	Bottom-up
bekannter Basiswert	früher return	vor den Schleifen in dp eintragen
ungültiger Zustand	passenden Wert zurückgeben	Zustand überspringen
nicht erreichbar	False/Unendlich zurückgeben	gleicher Startwert in dp

Basiswert wählen

Kleinstes direkt lösbares Teilproblem: leere Auswahl oft 0, beim Zählen oft 1, nicht erreichbar je nach Ziel False, inf oder -inf.

3.2.3. Vorgänger und Übergang bestimmen

Was aus der Rekurrenz übernommen wird

- **Vorgänger** — Alle kleineren Zustände, deren Werte für den aktuellen Zustand gebraucht werden.
- **Erlaubte Wahl** — Bedingungen aus der Aufgabe bleiben als if vor dem Aufruf beziehungsweise dp-Zugriff erhalten.
- **Verknüpfung** — Möglichkeiten mit max, min, logischem Oder oder Addition verbinden.

Derselbe Übergang in beiden Varianten

```
# Top-down: Vorgänger durch Aufruf holen
result = max(gain + solve(i-2, memo),
    solve(i-1, memo))
```

```
# Bottom-up: dieselben Vorgänger aus dp lesen
dp[i] = max(gain + dp[i-2],
    dp[i-1])
```

3.2.4. Reine Rekursion als Ausgangslage

Reine Rekursion zum Vergleich mit DP-Varianten

```
def solve(i, j):
    # Zustand (→ 3.2.1)
    if ...:
        # Basis (→ 3.2.2)
        return ...

    # Vorgänger & Übergang (→ 3.2.3)
    # Richtung: Top-down via Rekursionspfad (→ 3.2.7)
    # DP-Umbau: solve(prev) wird zu dp[prev] (Bottom-up)
    result = value + solve(prev_i, prev_j)
    return result
    # Ziel (→ 3.2.8)
```

3.2.5. Rekursion mit Memoization ergänzen

Rekurrenz unverändert lassen: Zustand (→ 3.2.1), Basis (→ 3.2.2) und Übergang (→ 3.2.3) übernehmen; nur Lookup und Speichern ergänzen.

Allgemeines Memoization-Gerüst

```
def solve(i, j, memo=None):
    if memo is None:
        # FEST: einmal erzeugen
        memo = {}

    state = (i, j)
    # Zustand (→ 3.2.1)
    if state in memo:
        # FEST: bereits berechnet?
        return memo[state]

    if ...:
        # Basis (→ 3.2.2)
        return ...

    # Vorgänger: Zustände, deren Werte wir brauchen
    # Übergang: rechte Seite der Rekurrenz übernehmen (→ 3.2.3)
    # Beispiel: Wert + Ergebnis des Vorgängers
    # bei JEDEM solve-Aufruf dasselbe memo mitgeben
    # if condition: # ANPASSEN: falls nötig; result einrücken
    result = value + solve(prev_i, prev_j, memo) # ANPASSEN
    memo[state] = result # FEST: vor return speichern
    return result # Ziel (→ 3.2.8)
```

3.2.6. Dieselbe DP Bottom-up aufbauen

Zustand, Basis und Übergang bleiben gleich. Aufrufe werden zu dp[prev]; dazu Füllreihenfolge (→ 3.2.7) wählen.

Allgemeines Bottom-up-Gerüst

```
def solve_bottom_up(target):
    dp = {}
    # FEST: Wert pro Zustand
    dp[...] = ... # Basis (→ 3.2.2)
    # STATT Rekursion: Vorgänger vor aktuellem Zustand
    for i, j in states_in_order: # Richtung (→ 3.2.7)
        state = (i, j)
        # Zustand (→ 3.2.1)
        if state in dp:
            # Basis nicht überschreiben
            continue

        # Vorgänger aus der Rekurrenz als dp-Schlüssel
        prev = (prev_i, prev_j)
        # dp[prev] liest seinen bereits berechneten Wert
        # if condition: # ANPASSEN: falls nötig; result einrücken
        result = value + dp[prev] # Übergang; ANPASSEN (→ 3.2.3)
        dp[state] = result # FEST: Ergebnis speichern
    return dp[target] # Ziel (→ 3.2.8)
```

3.2.7. Füllreihenfolge wählen

Füllrichtung aus den Vorgängern ablesen

Zustand braucht	Achse füllen
kleinere Indizes wie i-1, i-k	i aufsteigend
grössere Indizes wie i+1, i+k	i absteigend
(i-a, j-b)	betroffene Achsen aufsteigend
(i+a, j-b)	i absteigend, j aufsteigend

Top-down

Keine Schleifenrichtung: Der Zielaufruf folgt den Vorgängern rekursiv bis zur Basis. Bottom-up muss jeden gelesenen dp-Wert vorher berechnen.

3.2.8. Zielwert lesen und Varianten übersetzen

Dieselben Bausteine in beiden Varianten

Baustein	Memoization	Bottom-up
Zustand (→ 3.2.1)	Schlüssel state	Index/Schlüssel von dp
Basisfall (→ 3.2.2)	früher return	vorab in dp eintragen
Vorgänger (→ 3.2.3)	solve(prev, memo)	dp[prev]
Reihenfolge (→ 3.2.7)	rekursive Aufrufe	passende Schleifen
Ziel (→ 3.2.8)	solve(target)	dp[target]

Was bleibt, was ändert sich?

- **Bleibt gleich** — Bedeutung des Zustands, Basiswerte, Vorgänger, erlaubte Wahl, Verknüpfung und gesuchtes Ziel.
- **Ändert sich** — Memo-Lookup und rekursive Aufrufe werden zu vorgefüllten Basiswerten, dp-Zugriffen und Schleifen.
- **Zurück zu Top-down** — dp-Indizes werden Parameter, Basiszellen frühe Rückgaben und dp[prev] rekursive Aufrufe; Schleifen entfallen (→ 3.2.5).

3.3. Start hier: Aufgabe erkennen

- **FEST** — Grundstruktur der DP-Familie; normalerweise übernehmen.
- **ANPASSEN** — Diese Zeile hängt von Zustand, Basis, Übergang oder Ziel der Aufgabe ab.
- **OPTIONnnn** — Nummeriertes Zusatzfeature eines Skeletons. Nur für den erklärten Fall alle Zeilen mit derselben Nummer einkommentieren.

Was steht in der Aufgabe? → Direkt zum Beispiel

Typische Formulierung	Gesucht	Direkt zu
Rekursiver Code gegeben; Memoization oder Bottom-up verlangt	gleiche Lösung	3.2 Umbauen
Mathematische Rekurrenz gegeben; implementieren	Funktionswert	3.4.1 Rekurrenz
Pro Position wählen oder auslassen ; Nachbarn verboten	grösster Wert	3.4.2 House Robber
Gegenstände höchstens einmal wählen; Kapazitätslimit	grösster Wert	3.5.1 Knapsack
Zahlen höchstens einmal verwenden; Zielsumme erreichen	Ja / Nein	3.5.2 Subset Sum
Betrag mit denselben Münzen beliebig oft aufbauen	kleinste Anzahl	3.5.3 Coin Change
Anzahl verschiedener Münzkombinationen bestimmen	Anzahl	3.5.4 Kombinationen
Anzahl verschiedener Reihenfolgen bestimmen	Anzahl	3.5.4 Reihenfolgen
Länge in Stücke mit gegebenen Werten zerlegen	grösster Wert	3.5.5 Rod Cutting
Werte beliebig oft verwenden; erreichbare Beträge	Ja / Nein	3.5.6 Reachable
Zwei Strings oder Listen vergleichen	Länge	3.6.1 LCS-Länge
Konkrete gemeinsame Teilfolge ausgeben	Länge + Folge	3.6.1 LCS-Folge
Durch ein Gitter bewegen ; Werte maximieren, Zellen gesperrt	grösster Wert	3.7.1 Grid-Wert

Gesuchtes Resultat → Verknüpfung

Frage	Operation	Startwert
bester Wert	max	-float("inf")
kleinste Kosten	min	float("inf")
erreichbar?	logisches Oder	False; Start True
Anzahl Möglichkeiten	Addition	0; Start 1

3.4. 1D-DP: dp[i] — Lineare Sequenzen

Musterkarte

- **Ziel** — Für jede Position das beste Ergebnis bis dorthin speichern, damit die Lösung am Ende der Reihe feststeht.
- **Zustand** — dp[i] = Lösung bis Position beziehungsweise Grösse i.
- **Feste Vorgänger** — Nur frühere Werte wie i-1, i-2: 3.4.1 Fibonacci.
- **Nehmen/Auslassen** — Aktuelles Element wählen oder überspringen: 3.4.2 House Robber.
- **Aufwand** — Meist O(n) Zeit und O(n) Speicher.

3.4.1. Rekurrenzen / feste Vorgänger

Muster: mathematische Rekurrenz übertragen

- **Erkennen** — Eine Folge oder Funktion ist durch Basisfälle und eine Formel mit früheren Werten wie P(n-1) oder P(n-2) definiert.
- **Basisfälle** — Bekannte Werte direkt in memo beziehungsweise dp eintragen.
- **Übertragen** — Jedes P(n-k) wird zum rekursiven Aufruf; Faktoren, Rechenzeichen und Vergleiche unverändert übernehmen.
- **Top-down** — Wenn Rekursion oder Memoization verlangt ist: vom Ziel aus rechnen und jeden neuen Wert speichern.
- **Bottom-up** — Wenn Iteration erlaubt ist: ab den Basisfällen vorwärts rechnen.

Top-down: Memoization

```
def recurrence_memo(n, memo=None):  
    # ANPASSEN: alle Basisfälle aus der Aufgabe  
    if memo is None:  
        memo = {0: ..., 1: ...}  
  
    # FEST: bekannten Wert nicht erneut berechnen  
    if n in memo:  
        return memo[n]  
  
    # ANPASSEN: rechte Seite der Rekurrenz direkt einsetzen  
    # P(n-k) wird zu recurrence_memo(n-k, memo)  
    memo[n] = ...  
    return memo[n]
```

Memoization: typische Fehler

- **Gleicher Speicher** — In jedem rekursiven Aufruf dasselbe memo weitergeben.
- **Standardargument** — memo=None verwenden und das Dictionary erst in der Funktion erzeugen.
- **Formel** — Faktoren, Minuszeichen und Klammern aus der Rekurrenz nicht verlieren.
- **Reihenfolge** — Wert berechnen, in memo[n] speichern und danach zurückgeben.

Bottom-up: Rekurrenz vorwärts berechnen

```
def recurrence_bottom_up(n):  
    # ANPASSEN: alle Basisfälle aus der Aufgabe  
    dp = [None] * (n + 1)  
    dp[0] = ...  
    if n >= 1:  
        dp[1] = ...  
  
    # FEST: nach den Basisfällen vorwärts rechnen  
    for i in range(2, n + 1):  
        # ANPASSEN: P(i-k) wird zu dp[i-k]  
        dp[i] = ...  
    return dp[n]
```

3.4.2. House Robber / Coin Row

Muster: nehmen oder weglassen

- **Problem** — Keine zwei direkt benachbarten Werte gleichzeitig wählen; Summe maximieren.
- **Weglassen** — Bestes Ergebnis bis zum Vorgänger: dp[i-1].
- **Nehmen** — Aktueller Wert plus bestes Ergebnis vor dem verbotenen Abstand.
- **Anpassen** — min_distance=2 verbietet direkte Nachbarn; für grössere Sperrabstände erhöhen. Basis und max/min an die Aufgabe anpassen.

Maximaler Wert: Nehmen/Auslassen

```
def house_robber(nums, min_distance=2):  
    # nums[i] = Wert an Position i  
    num_positions = len(nums)  
    if num_positions == 0:  
        return 0  
  
    # FEST: Zustand und Basis für die ersten i Elemente  
    dp = [0] * (num_positions + 1)  
    dp[0], dp[1] = 0, nums[0]  
    # FEST: pro Position weglassen oder nehmen  
    for i in range(2, num_positions + 1):  
        skip = dp[i-1] # ANPASSEN: weglassen  
        previous = max(0, i-min_distance) # ANPASSEN: Abstand  
        take = dp[previous] + nums[i-1] # ANPASSEN: nehmen  
        dp[i] = max(skip, take) # ANPASSEN: Ziel  
    return dp[num_positions]
```

3.5. Kapazitäts- und Betrags-DP

Musterkarte

- **Ziel** — Mit den verfügbaren Objekten oder Beträgen den besten Wert finden oder einen gewünschten Betrag erreichen.
- **0/1-Auswahl** — Jedes Objekt höchstens einmal: 3.5.1 Knapsack.
- **0/1-Erreichbarkeit** — Jede Zahl höchstens einmal: 3.5.2 Subset Sum.
- **Wiederholbare Wahl** — Dieselbe Wahl beliebig oft: 3.5.3 Coin Change.
- **Wiederholbarer Wert** — Besten Wert mit beliebig oft nutzbaren Teilen: 3.5.5 Rod Cutting.
- **Wiederholbare Erreichbarkeit** — Erreichbare Beträge mit beliebig vielen Wiederholungen: 3.5.6 Reachable Amounts.

Eine Betragsachse: Was ändert sich?

Aufgabe	Nutzung	Kernänderung
0/1-Maximum	höchstens einmal	vorherige Tabellenzeile
0/1-erreichbar	höchstens einmal	Beträge rückwärts, Oder kleinere Beträge vorwärts,
Minimum	beliebig oft	min
erreichbar	beliebig oft	kleinere Beträge vorwärts, Oder
Anzahl Kombinationen	beliebig oft	Münzen aussen, Addition
Maximum	beliebig oft	kleinere Beträge vorwärts, max

3.5.1. 0/1 Knapsack

Muster: nehmen oder weglassen

- **Entscheidung** — Weglassen dp[i-1][w] oder Nehmen values[i-1] + dp[i-1][w-weights[i-1]].
- **Anpassen** — Gewichte, Werte, Basis, max/min; vorige Zeile macht Auswahl zu 0/1.

Maximaler Wert: 0/1-Auswahl

```
def knapsack(weights, values, capacity):  
    # weights[i] und values[i] gehören zum selben Objekt  
    num_items = len(values)  
    # FEST: Zustand und Basis für 0 Objekte  
    dp = [[0] * (capacity + 1)  
          for _ in range(num_items + 1)]  
    # FEST: jede Zelle aus Auslassen oder Nehmen bilden  
    for i in range(1, num_items + 1): # FEST: Objektachse  
        for w in range(capacity + 1): # FEST: Betragsachse  
            dp[i][w] = dp[i-1][w] # ANPASSEN: AUSLASSEN  
            if weights[i-1] <= w: # ANPASSEN: erlaubt?  
                dp[i][w] = max(  
                    dp[i][w],  
                    values[i-1] + dp[i-1][w-weights[i-1]]  
                ) # ANPASSEN: NEHMEN + Ziel  
    return dp[num_items][capacity] # maximaler Wert
```

3.5.2. Subset Sum: 0/1-Erreichbarkeit

Ja/Nein: jede Zahl höchstens einmal

```
def subset_sum(numbers, target):  
    # FEST: Boolean-Zustand; nur Summe 0 ist anfangs erreichbar  
    reachable = [False] * (target + 1)  
    reachable[0] = True  
    for number in numbers: # FEST: jede Zahl ansehen  
        # ANPASSEN: genau eine der beiden Richtungen wählen  
        amounts = range(target, number-1, -1) # 0/1: rückwärts  
        # amounts = range(number, target+1) # wiederholbar  
        for amount in amounts:  
            # ANPASSEN: weglassen ODER nehmen  
            reachable[amount] = (reachable[amount]  
                                or reachable[amount-number])  
    return reachable[target] # Ziel erreichbar?
```

DEV · 2026-09-15

3a

3.5.3. Coin Change: wiederholbare Wahl

Minimale Anzahl: wiederholbare Wahl

```
def coin_change(coin_values, target_amount):
    # coin_values = erlaubte Werte; target_amount = Ziel
    # FEST: Minimum-Zustand; Betrag 0 kostet keine Münze
    dp = [float("inf")] * (target_amount + 1)
    dp[0] = 0
    # FEST: Beträge vorwärts aus kleineren Beträgen bauen
    for amount in range(1, target_amount + 1):
        for coin in coin_values:
            # ANPASSEN: erlaubte Wahl
            if coin <= amount:
                # ANPASSEN: erlaubt?
                candidate = dp[amount-coin] + 1
                dp[amount] = min(dp[amount], candidate) # Ziel
    if dp[target_amount] == float("inf"):
        return -1
    return dp[target_amount] # kleinste Münzanzahl
```

3.5.4. Coin Change: Kombinationen oder Reihenfolgen

Entscheidender Unterschied

- **Kombinationen** — Münzen aussen: 1+2 und 2+1 zählen zusammen nur einmal.
- **Reihenfolgen** — Beträge aussen: 1+2 und 2+1 zählen als zwei Möglichkeiten.
- **Anpassen** — Erlaubte Werte, Zielbetrag, Basis ways[0]=1 und Addition.

Anzahl: Kombinationen oder Reihenfolgen

```
def count_coin_combinations(coin_values, target_amount):
    # FEST: Zählzustand; die leere Auswahl zählt für Betrag 0
    ways = [0] * (target_amount + 1)
    ways[0] = 1
    # FEST für Kombinationen: jede Münze als Typ öffnen
    for coin in coin_values:
        # FEST für Wiederholung: Beträge vorwärts
        for amount in range(coin, target_amount + 1):
            ways[amount] += ways[amount-coin] # ANPASSEN: zählen
    return ways[target_amount]
```

```
def count_coin_sequences(coin_values, target_amount):
    # FEST: gleicher Zählzustand, aber andere Schleifenreihenfolge
    ways = [0] * (target_amount + 1)
    ways[0] = 1
    # FEST für Reihenfolgen: jeden Betrag der Reihe nach öffnen
    for amount in range(1, target_amount + 1):
        for coin in coin_values:
            if coin <= amount:
                ways[amount] += ways[amount-coin] # ANPASSEN
    return ways[target_amount]
```

3.5.5. Rod Cutting: maximaler Wert

Muster: Teile beliebig oft verwenden

- **Zustand** — dp[length] = bester Wert für genau diese Gesamtlänge.
- **Übergang** — Passendes Stück anhängen: Stückwert plus bester Wert der Restlänge.
- **Anpassen** — Erlaubte Stücklängen, zugehörige Werte, Ziellänge und max/min.

Bester Wert: wiederholbare Stücke

```
def rod_cut(piece_lengths, piece_values, target_length):
    # FEST: Länge 0 hat Wert 0; andere noch unerreichbar
    dp = [-float("inf")] * (target_length + 1)
    dp[0] = 0

    # FEST: kleinere Längen vor grösseren berechnen
    for length in range(1, target_length + 1):
        for piece, value in zip(piece_lengths, piece_values):
            if piece <= length:
                candidate = value + dp[length-piece]
                dp[length] = max(dp[length], candidate)

    return dp[target_length] # maximaler Wert
```

3.5.6. Reachable Amounts: Boolean-DP

Ja/Nein: Werte beliebig oft

```
def reachable_amounts(coin_values, limit):
    # coin_values dürfen wiederholt werden; prüfe 0..limit
    # FEST: Boolean-Zustand; nur Betrag 0 ist anfangs erreichbar
    reachable = [False] * (limit + 1)
    reachable[0] = True
    # FEST: Beträge vorwärts aus kleineren Beträgen bauen
    for amount in range(1, limit + 1):
        for coin in coin_values:
            # ANPASSEN: erlaubte Wahl
            if amount >= coin and reachable[amount-coin]:
                reachable[amount] = True # ANPASSEN: erreichbar
                break
    return reachable # Boolean-Antwort für jeden Betrag
```

3.6. Zwei Präfixe: dp[i][j]

Musterkarte

- **Ziel** — Zwei Folgen schrittweise vergleichen und die beste gemeinsame Lösung für ihre Präfixe aufbauen.
- **Zustand** — dp[i][j] = Lösung für Präfix i von A und Präfix j von B.
- **Gleich** — Diagonal weitergehen; bei LCS kommt ein Treffer hinzu.
- **Ungleich** — Eine Seite auslassen und den besseren Vorgänger wählen.
- **Anpassen** — Basiszeile/-spalte sowie Übergänge bei Gleichheit und Ungleichheit bestimmen die konkrete Zwei-Präfix-Aufgabe.
- **OPTION01: Folge ausgeben** — Nur wenn die konkrete gemeinsame Folge verlangt ist: alle OPTION01-Zeilen einkommentieren und die Rekonstruktion verwenden.
- **Skeleton** — 3.6.1 LCS für zwei Strings oder Listen.

3.6.1. Longest Common Subsequence (LCS): Länge

Länge der LCS: Bottom-up-Skeleton

```
def lcs(sequence_a, sequence_b):
    # Eingaben sind zwei Strings oder Listen
    len_a, len_b = len(sequence_a), len(sequence_b)
    # FEST: Präfixzustand; ein leeres Präfix hat Wert 0
    dp = [[0] * (len_b + 1) for _ in range(len_a + 1)]
    # FEST: alle Präfixpaare zeilenweise füllen
    for i in range(1, len_a + 1):
        # FEST: Präfix A
        for j in range(1, len_b + 1):
            # FEST: Präfix B
            # ANPASSEN: gleich →diagonal; ungleich →oben/links
            if sequence_a[i-1] == sequence_b[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    # return dp[len_a][len_b], dp # OPTION01
    return dp[len_a][len_b] # Standard: nur Länge
```

OPTION01: konkrete LCS rekonstruieren

```
def reconstruct_lcs(sequence_a, sequence_b, dp):
    i, j = len(sequence_a), len(sequence_b)
    chosen = []
    # FEST: vom Ziel rückwärts bis ein Präfix leer ist
    while i > 0 and j > 0:
        if sequence_a[i-1] == sequence_b[j-1]:
            chosen.append(sequence_a[i-1]) # MATCH gewählt
            i, j = i - 1, j - 1
        elif dp[i-1][j] >= dp[i][j-1]:
            i -= 1 # A-Wert auslassen
        else:
            j -= 1 # B-Wert auslassen
    chosen.reverse()
    return chosen
```

3.7. Grid-DP: dp[i][j]

Musterkarte

- **Ziel** — Einen optimalen Weg durchs Gitter finden, der erlaubte Zellen nutzt und gesperrte Zellen meidet.
- **Zustand** — dp[row][col] = bester Wert vom Start bis zu dieser Zelle.
- **Schnell oder flexibel** — Wenige feste Vorgänger direkt einsetzen; bei variablen oder vielen Bewegungen den FLEX-Block mit moves verwenden.
- **Füllrichtung** — Spalten von links nach rechts, darin Zeilen von oben nach unten; jede Bewegung muss auf bereits berechnete Zellen zeigen.
- **Anpassen** — moves, blocked, start, Füllrichtung, Zellbeitrag, max/min und Zielauslese.
- **Grenze** — Erzeugen Bewegungen Zyklen, braucht es einen zusätzlichen Zustand oder einen Graphalgorithmus.

3.7.1. Grid-Wert: maximale Belohnung mit Sperren

Bottom-up-Skeleton: Initialisierung

ANPASSEN: Bewegung = (Zeilenänderung, Spaltenänderung)
moves = [(1, 0), (0, 1)] # unten, rechts

```
def grid_dp(grid, moves, blocked=-1, start=(0, 0)):
    rows, cols = len(grid), len(grid[0])
    start_row, start_col = start # ANPASSEN: Start
    # FEST: bester Wert vom Start bis zu jeder Zelle
    # None bedeutet: Zelle bisher nicht erreichbar
    dp = [[None] * cols for _ in range(rows)]
    # ANPASSEN: erreichbaren Start als Basis setzen
    if grid[start_row][start_col] != blocked:
        dp[start_row][start_col] = grid[start_row][start_col]

    # Bottom-up-Skeleton: Vorgänger & Füllen
    # ANPASSEN: Füllrichtung muss zu moves passen
    for col in range(cols):
        # links →rechts
        for row in range(rows):
            # oben →unten
            if (row, col) == start or grid[row][col] == blocked:
                continue
            prev = [] # erreichbare Vorgänger
            # FLEX: moves rückwärts →mögliche Vorgänger
            for row_change, col_change in moves:
                prev_row = row - row_change
                prev_col = col - col_change
                inside = (0 <= prev_row < rows
                        and 0 <= prev_col < cols)
                if inside and dp[prev_row][prev_col] is not None:
                    prev.append((prev_row, prev_col))
            if not prev:
                continue # bleibt unerreichbar
            # ANPASSEN: max für Belohnung, min für Kosten
            # key vergleicht die dp-Werte der Vorgänger
            best = max(
                prev,
                key=lambda pos: dp[pos[0]][pos[1]])
            prev_row, prev_col = best
            dp[row][col] = (dp[prev_row][prev_col]
                          + grid[row][col]) # ANPASSEN: Übergang
            # Standard: nur Werte

    return dp
```

SCHNELL: ersetzt nur den FLEX-Block

Beispiel unten/rechts: feste Vorgänger oben/links
if row > 0 and dp[row-1][col] is not None:
 prev.append((row-1, col)) # von oben
if col > 0 and dp[row][col-1] is not None:
 prev.append((row, col-1)) # von links

Ziel auslesen

Eine Zielzelle
return dp[target_row][target_col]

Mehrere mögliche Ziele
values = [dp[row][col] for row, col in targets
 if dp[row][col] is not None]
return max(values) # ANPASSEN: max oder min

4. Algorithmen: Sortieren

4.1. Übersicht: Sortier-Laufzeiten

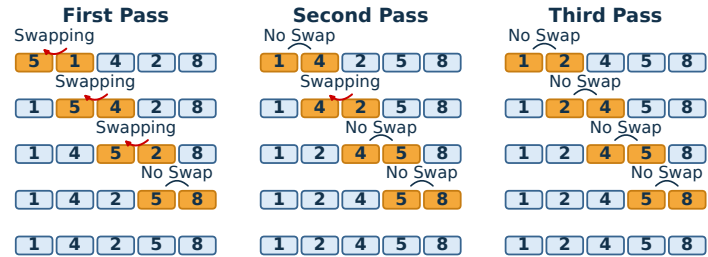
Algorithmus	Durchschnitt	Worst-Case
Bubble Sort	$\Theta(n^2)$	$\Theta(n^2)$
Selection Sort	$\Theta(n^2)$	$\Theta(n^2)$
Insertion Sort	$\Theta(n^2)$ (best: $\Theta(n)$)	$\Theta(n^2)$
Merge Sort	$\Theta(n \log n)$	$\Theta(n \log n)$
Quick Sort	$\Theta(n \log n)$	$\Theta(n^2)$
Heap Sort	$\Theta(n \log n)$	$\Theta(n \log n)$
Binary Insertion Sort	$\Theta(n^2)$	$\Theta(n^2)$

Details: Vergleiche und Vertauschungen

sort	# swaps			# comparisons			general
	case	A	runtime	case	A	runtime	
Bubble	worst	reverse sorted	$\Theta(n^2)$	all	-	$\Theta(n^2)$	$\Theta(n^2)$
	average	-	$\Theta(n^2)$				
	best	sorted	$\Theta(1)$ (0)				
Selection	worst	reverse sorted	$\Theta(n)$	all	-	$\Theta(n^2)$	$\Theta(n^2)$
	average	-	$\Theta(n)$				
	best	sorted	$\Theta(1)$ (0)				
Insertion	worst	reverse sorted	$\Theta(n^2)$	worst	reverse sorted	$\Theta(n^2)$	$\Theta(n^2)$
	average	-	$\Theta(n^2)$	average	-	-	$\Theta(n^2)$
	best	sorted	$\Theta(1)$ (0)	best	sorted	$\Theta(n)$	$\Theta(n)$
Merge	all	-	$\Theta(n \log(n))$	all	-	$\Theta(n \log(n))$	$\Theta(n)$
	Braucht zusätzlich $\Theta(n)$ Speicher für die Subarrays						
Quick	worst	*	$\Theta(n \log(n))$	worst	*	$\Theta(n^2)$	$\Theta(n^2)$
	average	**	$\Theta(n \log(n))$	average	**	$\Theta(n \log(n))$	$\Theta(n \log(n))$
	best	***	$\Theta(n)$	best	***	$\Theta(n \log(n))$	$\Theta(n \log(n))$
Heap	all	-	$\Theta(n \log(n))$	all	-	$\Theta(n \log(n))$	$\Theta(n \log(n))$

4.2. Quadratische Sortierverfahren

4.2.1. Bubble Sort



Ablauf

1. Vergleichen & Tauschen

Vergleicht benachbarte Elemente und vertauscht sie bei falscher Reihenfolge.

2. Wiederholen

Wiederholt dies, bis keine Vertauschungen mehr nötig sind.

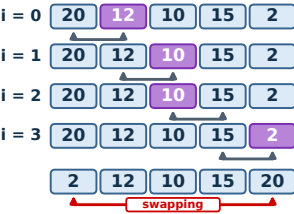
- Invariante** — $a[:-(i + 1)]$ enthält die $i + 1$ grössten Elemente von a in sort. Reihe.
- Laufzeit** — $\Theta(n^2)$ im Durchschnitt und schlechtesten Fall.

Bubble Sort (In-Place, mutiert Array)

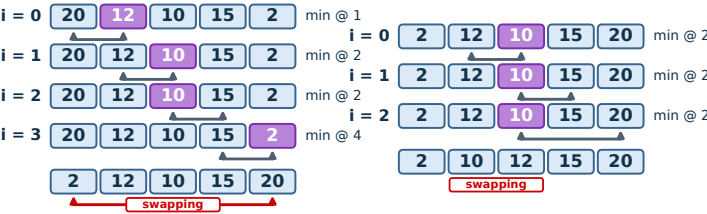
```
def bubble_sort(a):  
    n = len(a)  
    # Grosse Schleife über alle Elemente  
    for i in range(n):  
        # Vergleicht bis zum unsortierten Rest  
        for j in range(0, n-i-1):  
            # Wenn linkes Element grösser...  
            if a[j] > a[j+1]:  
                # ...vertausche beide  
                a[j], a[j+1] = a[j+1], a[j]
```

4.2.2. Selection Sort

step = 0



step = 1



Ablauf

1. Minimum suchen

Sucht in jedem Durchlauf das kleinste Element.

2. Tauschen

Tauscht es mit dem aktuellen Element.

3. Wiederholen

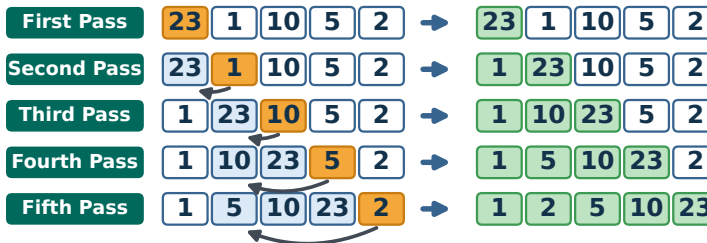
Wiederholt das für den restlichen unsortierten Bereich.

- Invariante** — $a[:i + 1]$ enthält die kleinsten $i + 1$ Elemente von a , in sort. Reihe.
- Laufzeit** — $\Theta(n^2)$ in allen Fällen.

Selection Sort (In-Place, mutiert Array)

```
def selection_sort(a):  
    n = len(a)  
    # Grosse Schleife über alle Positionen  
    for i in range(n):  
        # Nimm an, dass a[i] das Minimum ist  
        min_idx = i  
        # Suche im restlichen Feld...  
        for j in range(i+1, n):  
            # ...nach einem noch kleineren Element  
            if a[j] < a[min_idx]:  
                # Speichere neuen Index des Minimums  
                min_idx = j  
        # Tausche an sortierte Position  
        a[i], a[min_idx] = a[min_idx], a[i]
```

4.2.3. Insertion Sort



Ablauf

1. Idee

Sortiert ähnlich wie beim Kartenspiel.

2. Einfügen

Jedes Element wird an die richtige Position im linken, bereits sortierten Teil eingefügt.

3. Verschieben

Grössere Elemente werden nach rechts verschoben.

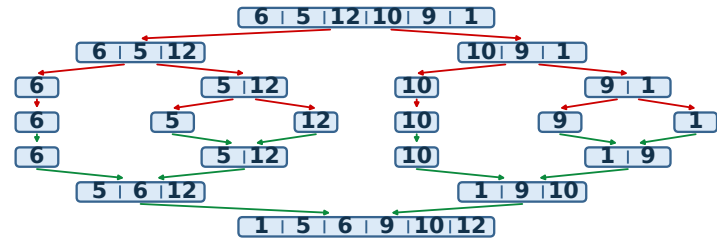
- Invariante** — Die Elemente in $a[:i + 1]$ sind sortiert und entsprechen den ersten i Elementen des ursprüngl. Arrays.
- Laufzeit** — $\Theta(n)$ im besten Fall (wenn sortiert), $\Theta(n^2)$ im Worst-Case.

def insertion_sort(a):

```
# Starte beim zweiten Element  
for i in range(1, len(a)):  
    # Speichere aktuelles Element  
    key = a[i]  
    # Betrachte die Elemente links davon  
    j = i - 1  
    # Solange links grössere Elem. sind  
    while j >= 0 and key < a[j]:  
        # Verschiebe sie nach rechts  
        a[j + 1] = a[j]  
        # Gehe weiter nach links  
        j -= 1  
    # Füge das Element an Lücke ein  
    a[j + 1] = key
```

4.3. Divide-and-Conquer-Sortierung

4.3.1. Merge Sort



Ablauf

1. Idee

Rekursiver Divide-and-Conquer-Algorithmus.

2. Teilen

Das Array wird in zwei Hälften geteilt.

3. Rekursiv sortieren

Beide Hälften werden rekursiv sortiert.

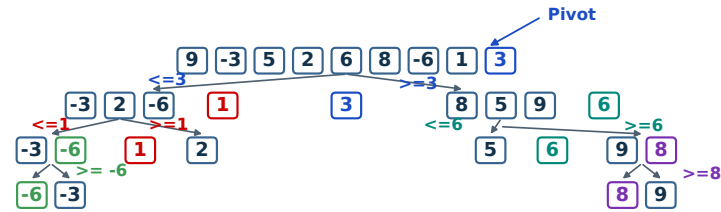
4. Zusammenführen

Die sortierten Hälften werden zusammengeführt (merge).

- Invariante** — Das Resultat-Array $b[:i + 1]$ enthält die $i + 1$ kleinsten Elemente von a in sortierter Reihenfolge.
- Laufzeit** — $\Theta(n \log n)$ in allen Fällen.

```
def merge_sort(a):  
    # Wenn Array mehr als 1 Element hat  
    if len(a) > 1:  
        # Finde die Mitte  
        mid = len(a) // 2  
        # Teile das Array in zwei Hälften  
        L, R = a[:mid], a[mid:]  
        # Sortiere beide rekursiv  
        merge_sort(L); merge_sort(R)  
        i = j = k = 0  
        # Beide Hälften zusammenfügen  
        while i < len(L) and j < len(R):  
            # Nimm das kleinere Element ...  
            if L[i] < R[j]:  
                # ...aus linker Hälfte  
                a[k] = L[i]; i+=1  
            else:  
                # ...aus rechter Hälfte  
                a[k] = R[j]; j+=1  
            k+=1  
        # Rest links anhängen  
        while i < len(L):  
            a[k] = L[i]; i+=1; k+=1  
        # Rest rechts anhängen  
        while j < len(R):  
            a[k] = R[j]; j+=1; k+=1
```

4.3.2. Quick Sort



Ablauf

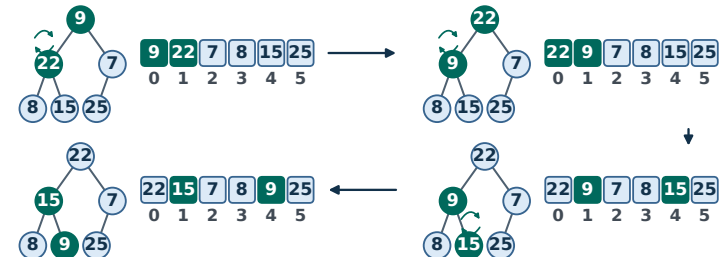
- Idee**
Effizienter rekursiver Divide-and-Conquer-Algorithmus.
- Pivot wählen**
Wählt ein Pivot-Element.
- Partitionieren**
Teilt das Array in Elemente kleiner und grösser als das Pivot.
- Rekursiv sortieren**
Sortiert beide Teilarrays rekursiv.

- Invariante** — Das Pivotelement befindet sich an der richtigen Posit., Elem. links davon sind kleiner und rechts davon grösser als das Pivot.
- Laufzeit** — $\mathcal{O}(n \log n)$ durchschnittlich, $\mathcal{O}(n^2)$ im Worst-Case.

```
def quick_sort(a, low, high):  
    # Wenn mehr als ein Element da ist  
    if low < high:  
        # Wähle ganz rechts als Pivot  
        pivot = a[high]  
        # Zeiger für kleineres Element  
        i = low - 1  
        # Gehe durch das Array  
        for j in range(low, high):  
            # Wenn Element kleiner/gleich Pivot  
            if a[j] <= pivot:  
                # Verschiebe Zeiger  
                i += 1  
                # Vertausche Elemente  
                a[i], a[j] = a[j], a[i]  
        # Setze Pivot in die Mitte  
        a[i+1], a[high] = a[high], a[i+1]  
        # Dies ist die fertige Pivot-Position  
        pi = i + 1  
        # Sortiere rekursiv links vom Pivot  
        quick_sort(a, low, pi - 1)  
        # Sortiere rekursiv rechts vom Pivot  
        quick_sort(a, pi + 1, high)
```

4.4. Heap-basierte Sortierung

4.4.1. Heap Sort



Ablauf

- Max-Heap aufbauen**
Erst wird ein Max-Heap aufgebaut. ($\rightarrow$ 6.5)
- Wurzel entnehmen**
Grösstes Element (Wurzel) wird entfernt und ans Ende gesetzt.
- Heapify**
Der Heap wird danach angepasst (heapify).

- Laufzeit** — $\mathcal{O}(n \log n)$ in allen Fällen.

Heapify (Max-Heap)

```
def heapify(a, n, i):  
    # Setze root als grösstes Element  
    largest = i  
    # Linkes und rechtes Kind  
    l, r = 2 * i + 1, 2 * i + 2  
    # Kind links grösser?  
    if l < n and a[l] > a[largest]: largest = l  
    # Kind rechts grösser?  
    if r < n and a[r] > a[largest]: largest = r  
    # Wenn Root nicht das Grösste ist...  
    if largest != i:  
        # ...vertausche beide  
        a[i], a[largest] = a[largest], a[i]  
        # Max-Heap-Eigenschaft wiederherst.  
        heapify(a, n, largest)
```

Heap Sort

```
def heap_sort(a):  
    n = len(a)  
    # Baue einen Max-Heap auf (bottom-up)  
    for i in range(n//2 - 1, -1, -1):  
        heapify(a, n, i)  
    # Elemente einzeln entnehmen  
    for i in range(n-1, 0, -1):  
        # Aktuelle Wurzel ans Ende setzen  
        a[i], a[0] = a[0], a[i]  
        # Erneut Max-Heap aufbauen  
        heapify(a, i, 0)
```

4.5. Binäre Suche beim Einfügen

4.5.1. Binary Insertion Sort

1 Ausgangslage

Linker Teil bereits sortiert



2 Binäre Suche

Einfügeposition im sortierten Teil finden



7 > 5, dann 7 < 8
Position = 2

3 Platz schaffen und einfügen

Grössere Elemente nach rechts verschieben

Suche: $\mathcal{O}(\log n)$ Verschieben: $\mathcal{O}(n)$



Ablauf

- Idee**
Eine Variante von Insertion Sort.
- Binär suchen**
Findet die Einfügeposition per binärer Suche.
- Verschieben**
Die Verschiebung der Elemente bleibt aber linear.

- Laufzeit** — $\mathcal{O}(n^2)$ (Vergleiche: $\mathcal{O}(n \log n)$, Verschiebungen: $\mathcal{O}(n^2)$).

def binary_insertion_sort(a):

```
# Starte beim zweiten Element  
for i in range(1, len(a)):  
    key = a[i] # Zu fügendes Element  
    # --- 1. Binäre Suche nach Einfügeposition ---  
    low, high = 0, i - 1  
    while low <= high:  
        mid = (low + high) // 2  
        # Suche in linker Hälfte  
        if key < a[mid]: high = mid - 1  
        # Suche in rechter Hälfte  
        else: low = mid + 1  
    # --- 2. Elemente nach rechts verschieben ---  
    for j in range(i, low, -1):  
        a[j] = a[j - 1] # Eins nach rechts  
    a[low] = key # Setze an gefundene Pos.
```

5. Asymptotische Laufzeit

5.1. $\mathcal{O}$ -Grundregeln

Intuitive Bedeutung für MC

- $f(n) \in \mathcal{O}(g(n))$ — Höchstens so schnell wie $g(n)$ ($\leq$). g ist eine obere Schranke.
- $f(n) \in \mathcal{\Omega}(g(n))$ — Mindestens so schnell wie $g(n)$ ($\geq$). g ist eine untere Schranke.
- $f(n) \in \mathcal{\Theta}(g(n))$ — Exakt so schnell wie $g(n)$ ($=$). Beide haben dieselbe Klasse.
- $f(n) \in \mathcal{o}(g(n))$ — Echt langsamer als $g(n)$ ($<$).
- $f(n) \in \omega(g(n))$ — Echt schneller als $g(n)$ ($>$).

Rechenregeln für MC-Aufgaben

- Dominanz (Summenregel)** — Der am schnellsten wachsende Term gewinnt und bestimmt die Klasse. Langsamere Terme und Konstanten fallen weg.
Beispiel: $ 3n^2 + n \log n + 100 \in \mathcal{O}(n^2)$.
- Produktregel** — Klassen werden multipliziert: $\mathcal{O}(f(n)) \cdot \mathcal{O}(g(n)) = \mathcal{O}(f(n) \cdot g(n))$.
Beispiel: $ n \cdot \mathcal{O}(\log n) = \mathcal{O}(n \log n)$.
- Transitivität** — Wenn $f \in \mathcal{O}(g)$ und $g \in \mathcal{O}(h)$, dann ist $f \in \mathcal{O}(h)$. (Gilt für alle Klassen).
- Komplementarität** — $f(n) \in \mathcal{O}(g(n)) \iff g(n) \in \mathcal{\Omega}(f(n))$.

5.2. Wachstumsordnung

Die Master-Sortierliste (von langsam nach schnell)

$1 < \log(\log n) < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < 3^n < n! < n^n$
Jede noch so kleine Potenz $n^{0.01}$ wächst asymptotisch schneller als jeder Logarithmus $(\log n)^{100}$. Jeder Exponent 1.01^n wächst asymptotisch schneller als jedes Polynom n^{1000} .

Grenzwert-Kriterium für MC

Um zwei Funktionen $f(n)$ und $g(n)$ für $n \rightarrow \infty$ zu vergleichen:

Grenzwert $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$	Bedeutung	Ergebnis
0	f wächst echt langsamer $\mathcal{O}(g(n))$	$f(n) \in \mathcal{o}(g(n)) \Rightarrow f(n) \in \mathcal{O}(g(n))$
$c > 0$ (Konstante)	beide wachsen gleich schnell $\mathcal{\Theta}(f(n))$	$f(n) \in \mathcal{\Theta}(g(n)) \iff g(n) \in \mathcal{\Theta}(f(n))$
∞	f wächst echt schneller $\mathcal{\Omega}(g(n))$	$f(n) \in \omega(g(n)) \Rightarrow f(n) \in \mathcal{\Omega}(g(n))$

Schnell-Checks für Exponenten & Logarithmen

- Faktoren im Exponent ändern die Klasse:** $2^{2n} = 4^n \Rightarrow \mathcal{O}(4^n) \neq \mathcal{O}(2^n)$.
- Summanden im Exponent ändern die Klasse NICHT:** $2^{n+3} = 8 \cdot 2^n \Rightarrow \mathcal{O}(2^n)$.
- Logarithmus-Basen sind egal:** $\log_2 n \in \mathcal{O}(\log_{10} n)$ (Unterschied ist nur ein konstanter Faktor!).
- Exponent-Basen sind extrem wichtig:** $2^n \neq \mathcal{O}(3^n)$. Allgemein gilt $a^n < b^n$ für $a < b$.
- Logarithmus im Exponent:** $2^{\log_2 n} = n$. Aber Achtung bei anderer Basis: $2^{\log_3 n} = n^{\log_3 2} \approx n^{0.63}$ (wächst langsamer als $n!$).

5.3. Standardumformungen

MC-Tricks zur Termvereinfachung

- Exponenten-Tausch — $a^{\log_b c} = c^{\log_b a}$ **Der wichtigste MC-Trick!** Bringt die Variable aus dem Exponenten in die Basis.
Beispiel: $ 3^{\log_2 n} = n^{\log_2 3} \approx n^{1.58}$.
- Logarithmus-Regeln — $\log(a^k) = k \log a$.
Beispiel: $ \log(n^5) = 5 \log n \in \mathcal{O}(\log n)$.
- Fakultät mit Stirling — $n! \approx \left(\frac{n}{e}\right)^n \Rightarrow \log(n!) \in \mathcal{O}(n \log n)$.
- Logarithmus-Summe — $\log(a \cdot b) = \log a + \log b$.
- Zweierpotenzen im Logarithmus — $2^{k \log_2 n} = (2^{\log_2 n})^k = n^k$.

L'Hôpital-Ableitungsregel (falls Grenzwert unklar)

1. Ableitungen bilden

Ersetze n durch x und bilde die kontinuierlichen Ableitungen $f'(x)$ und $g'(x)$.

2. Grenzwert der Ableitung berechnen

Bestimme $L = \lim_{x \rightarrow \infty} \frac{f'(x)}{g'(x)}$.

3. Übertragen

Es gilt $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = L$.
Beispiel: $ \lim \frac{\ln n}{n} \rightarrow \lim \frac{1/x}{1} = 0 \Rightarrow \ln n < n$.

5.4. Häufige Summen		
Summen-Verhalten auf einen Blick		
Summe	Formel / Näherung	Laufzeit $\mathcal{O}$ & MC-Intuition
$\sum_{i=1}^n 1$	n	$\mathcal{O}(n)$ (Konstante Arbeit n -mal addieren)
$\sum_{i=1}^n i$	$\frac{n(n+1)}{2}$	$\mathcal{O}(n^2)$ (Arithmetische Summe, Dreiecksfläche)
$\sum_{i=1}^n i^2$	$\frac{n(n+1)(2n+1)}{6}$	$\mathcal{O}(n^3)$
$\sum_{i=1}^n i^k$	$\approx \frac{n^{k+1}}{k+1}$	$\mathcal{O}(n^{k+1})$ (Integral-Approximation: $\int x^k \in \mathcal{O}(x^{k+1})$)
$\sum_{i=0}^n q^i$ ($q > 1$)	$\frac{q^{n+1}-1}{q-1}$	$\mathcal{O}(q^n)$ (Geometrisch wachsend: dominiert durch das letzte Glied)
$\sum_{i=0}^n q^i$ ($q < 1$)	$\leq \frac{1}{1-q}$	$\mathcal{O}(1)$ (Geometrisch fallend: konvergiert gegen Konstante)
$\sum_{i=1}^n \frac{1}{i}$	$\ln n + \gamma \approx \ln n$	$\mathcal{O}(\log n)$ (Harmonische Reihe, extrem wichtig für Teiler)
$\sum_{i=1}^{\log n} i$	$\frac{(\log n)^2 + \log n}{2}$	$\mathcal{O}((\log n)^2)$
$\sum_{i=1}^n \log i$	$\log(n!) \approx n \log n$	$\mathcal{O}(n \log n)$
$\sum_{i=1}^n \sqrt{i}$	$\approx \frac{2}{3} n^{3/2}$	$\mathcal{O}(n^{3/2})$
$\sum_{i=1}^n i \log i$	keine einfache Formel	$\mathcal{O}(n^2 \log n)$
$\sum_{i=1}^n 2^{n-i}$	$2^n \cdot \sum_{i=1}^n 2^{-i} \approx 2^n$	$\mathcal{O}(2^n)$
$\sum_{i=1}^{\log n} 2^i$	$2^{\log n+1} - 1 = 2n - 1$	$\mathcal{O}(n)$ (Geometrische Summe bis $\log n$, verdoppelt sich)
$\sum_{i=1}^n \frac{n}{i}$	$n \cdot \sum_{i=1}^n \frac{1}{i}$	$\mathcal{O}(n \log n)$ (n wird aus der Summe ausgeklammert)

5.5. Schleifen als Summen

Kochrezept: Schleifen in Summen übersetzen

1. Äussere Schleife aufschreiben

Bestimme die Grenzen der äusseren Schleife (z.B. von 1 bis n) und schreibe diese als Summe: $\sum_{i=1}^n$.

2. Innere Schleife aufschreiben

Bestimme die Grenzen der inneren Schleife in Abhängigkeit der äusseren Laufvariable (z.B. von 1 bis i^2) und hänge diese an: $\sum_{i=1}^n \sum_{j=1}^{i^2}$.

3. Inhalt modellieren

Ersetze den Rumpf durch 1 (sofern er $\mathcal{O}(1)$ benötigt): $\sum_{i=1}^n \sum_{j=1}^{i^2} 1$.

4. Nachschlagen & Auflösen

Löse die innerste Summe auf und schlage das Ergebnis in der Summen-Tabelle 5.4 nach:
 $\sum_{i=1}^n (\text{innere Summe}) = \sum_{i=1}^n i^2 \in \mathcal{O}(n^3)$.

5.6. Typische Schleifenmuster

Platzhalter: $a(n)$ ist die äussere Grenze, $b(n)$ eine von i unabhängige innere Grenze und $b(i, n)$ eine von i abhängige innere Grenze. c, p, q sind Konstanten.

5.6.1. Unabhängige und konstante Grenzen

Eine Schleife

```
for i in range(a(n)):
    f()
```

 $\mathcal{O}(a(n))$

Unabhängige Grenzen

```
for i in range(a(n)):
    for j in range(b(n)):
        f()
```

 $\mathcal{O}(a(n) b(n))$

Konstante innere Grenze

```
for i in range(a(n)):
    for j in range(c):
        f()
```

 $\mathcal{O}(a(n))$

5.6.2. Abhängige innere Grenzen

Wachsende innere Grenze

```
for i in range(a(n)):
    for j in range(i):
        f()
```

 $\mathcal{O}(a(n)^2)$

Schrumpfende innere Grenze

```
for i in range(a(n)):
    for j in range(i, a(n)):
        f()
```

 $\mathcal{O}(a(n)^2)$

Dreifach gestaffelte Grenzen

```
for i in range(a(n)):
    for j in range(i):
        for k in range(j):
            f()
```

 $\mathcal{O}(a(n)^3)$

5.6.3. Multiplikative Updates

Durch Faktor teilen

$i = a(n)$
while $i > c$:
 $f()$
 $i //= q$
 $\mathcal{O}(\log a(n))$

Mit Faktor multiplizieren

$i = c$
while $i < a(n)$:
 $f()$
 $i *= q$
 $\mathcal{O}(\log a(n))$

Laufvariable potenzieren

$i = c$
while $i < a(n)$:
 $f()$
 $i = i ** p$
 $\mathcal{O}(\log \log a(n))$

5.6.4. Potenzen in Grenzen und Bedingungen

Potenz in der Abbruchbedingung

$m = 0$
while $m ** p < a(n)$:
 $f()$
 $m += 1$
 $\mathcal{O}(a(n)^{1/p})$

Ausdruck als äussere Grenze

for i **in** $\text{range}(a(n))$:
 for j **in** $\text{range}(i)$:
 $f()$
 $\mathcal{O}(a(n)^2)$

Potenz als innere Grenze

for i **in** $\text{range}(a(n))$:
 for j **in** $\text{range}(i ** p)$:
 $f()$
 $\mathcal{O}(a(n)^{p+1})$

5.6.5. Zusammengesetzte Grenzen

Teilerprüfung: gesamte Laufzeit

for i **in** $\text{range}(a(n))$:
 for j **in** $\text{range}(1, i + 1)$:
 if $i \% j == 0$:
 $f()$
 $\mathcal{O}(a(n)^2)$

Summe zweier Laufvariablen

for i **in** $\text{range}(a(n))$:
 for j **in** $\text{range}(a(n))$:
 for k **in** $\text{range}(i + j)$:
 $f()$
 $\mathcal{O}(a(n)^3)$

5.7. Rekurrenzen und Master-Theorem

Das Einmaleins des Master-Theorems

Für Rekurrenzen der Form: $T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$ ($a \geq 1, b > 1$)

- $T(n)$ — Gesamtlaufzeit bei Eingabegrösse n
- a — Anzahl der rekursiven Teilprobleme (Verzweigungsfaktor, $a \geq 1$)
- b — Faktor, um den die Problemgrösse geteilt wird ($b > 1$)
- $f(n)$ — Kosten für Aufteilung & Zusammenführung (äussere Arbeit)

Wir vergleichen die **Arbeit an den Blättern** $n^{\log_b a}$ mit der **äusseren Arbeit** $f(n)$:

Die drei Hauptfälle

Fall	Bedingung für $f(n)$	Wer gewinnt? (Dominanz)	Laufzeit $T(n)$
1	$f(n)$ ist polynomiell kleiner	Blätter gewinnen ($n^{\log_b a}$)	$\mathcal{O}(n^{\log_b a})$
2	$f(n)$ ist gleich gross	Unentschieden (Balance)	$\mathcal{O}(n^{\log_b a} \log n)$
3	$f(n)$ ist polynomiell grösser	Äussere Arbeit gewinnt ($f(n)$)	$\mathcal{O}(f(n))$

Hinweis: „Polynomiell„ bedeutet, der Unterschied muss mindestens Faktor n^ϵ ($\epsilon > 0$) sein. Ein blosser $\log n$ -Faktor reicht nicht für Fall 1 oder 3!

Kochrezept: Ausrollen für Nicht-Standard-Rekursionen

1. Einsetzen (Ausrollen)

Ersetze den rekursiven Term mehrmals durch sich selbst, um das Muster zu erkennen:
 $T(n) = T(n - 1) + n = T(n - 2) + (n - 1) + n = T(n - k) + \sum_{i=0}^{k-1} (n - i)$.

2. Abbruch finden

Stelle die Abbruchbedingung auf. Meist wenn das Argument 1 wird: $n - k = 1 \Rightarrow k = n - 1$.

3. Summe aufstellen

Ersetze k in der Summe und setze die Basis (z.B. $T(1) = c$) ein:
 $T(n) = T(1) + \sum_{j=2}^n j = c + \frac{n(n+1)}{2} - 1$.

4. Klasse ablesen

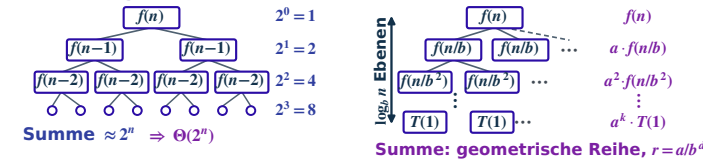
Asymptotischen Rumpf ablesen: $T(n) \in \mathcal{O}(n^2)$.

Beliebte MC-Rekursionen auf einen Blick

- $T(n) = T(n-1) + 1 \Rightarrow \mathcal{O}(n)$ (Konstante Arbeit auf jeder Stufe)
- $T(n) = T(n-1) + n \Rightarrow \mathcal{O}(n^2)$ (Lineare Arbeit auf jeder Stufe, arithmetische Reihe)
- $T(n) = T(n/2) + 1 \Rightarrow \mathcal{O}(\log n)$ (Binäre Suche)
- $T(n) = 2T(n/2) + n \Rightarrow \mathcal{O}(n \log n)$ (Merge Sort)
- $T(n) = 2T(n-1) + 1 \Rightarrow \mathcal{O}(2^n)$ (Fibonacci-artig, exponentieller Baum)

Rekursionsbaum-Modell

Verdopplung pro Ebene Aufrufe **Unroll:** $T(n) = aT(n/b) + f(n)$ **Kosten/Ebene**



Typische rekursive Implementierungen

1. Zwei rekursive Aufrufe (z.B. Fibonacci)

```
def fib(n):  
    if n <= 1: return n  
    return fib(n-1) + fib(n-2) # -> Theta(phi^n) ~ Theta(1.618^n)
```

Viele rekursive Aufrufe

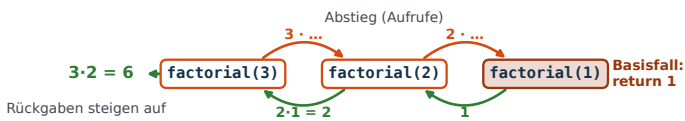
2. n-fache Rekursion (Fakultäts-Aufrufe)

```
def f(n):  
    if n <= 1: return  
    for i in range(n):  
        f(n-1) # -> Theta(n!)
```

Einfache rekursive Halbierung

```
def h(n):  
    if n <= 1: return  
    h(n // 2) # -> Theta(log n)
```

Rekursion und Rückgaben



6. Datenstrukturen

6.1. Übersicht: Laufzeiten im Vergleich

Datenstruktur	Suche	Einfügen	Entfernen
Liste / Array	$\mathcal{O}(n)$	$\mathcal{O}(n)$ (worst)	$\mathcal{O}(n)$ (worst)
Linked List	$\mathcal{O}(n)$	$\mathcal{O}(1)$ (nach Suche)	$\mathcal{O}(1)$ (nach Suche)
BST (balanciert)	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
BST (unbal.)	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Heap	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$ (Top)
Hash-Tabelle	$\mathcal{O}(1)$ (erw.)	$\mathcal{O}(1)$ (erw.)	$\mathcal{O}(1)$ (erw.)
Quadtree	$\mathcal{O}(\log n)$ (Range)	$\mathcal{O}(\log n)$	—

Merke

- Hash-Tabelle** — schnellster Zugriff im Durchschnitt, aber ungeordnet.
- BST / Heap** — geordneter Zugriff, log. Laufzeit — nur balanciert effizient.
- Quadtree** — spezialisiert auf 2D-Bereichs-Abfragen.

6.2. Queue und Stack

Queue

Eine Warteschlange, bei der Elemente nach dem **First In – First Out (FIFO)** Prinzip verarbeitet werden.

- add/enqueue** — Element am Ende der Queue einfügen
- remove/dequeue** — Element am Anfang der Queue entfernen

Stack

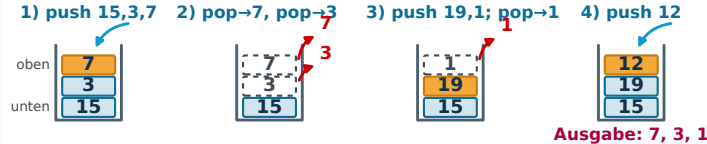
Ein Stapelspeicher, der nach dem **Last In – First Out (LIFO)** Prinzip arbeitet.

- add/push** — Element oben auf den Stack legen
- remove/pop** — Oberstes Element entfernen

Stack und Queue in Python

```
stack = [] # Stapel  
stack.append(x) # push: oben drauflegen -> O(1)  
top = stack.pop() # pop: oberstes wegnehmen -> O(1)  
  
from collections import deque # Queue  
q = deque() # doppelt verkettete Liste (O(1) an beiden Enden)  
q.append(x) # enqueue: hinten anstellen -> O(1)  
first = q.popleft() # dequeue: vorne entnehmen -> O(1)  
# list.pop(0) verschiebt alle = O(n) -> deque nutzen!
```

Stack: Push und Pop



Komplexitäten

- push / pop (Stack)** — $\mathcal{O}(1)$
- enqueue / dequeue (Queue)** — $\mathcal{O}(1)$ (mit deque, nicht list.pop(0))

6.3. Linked Lists

Linked List

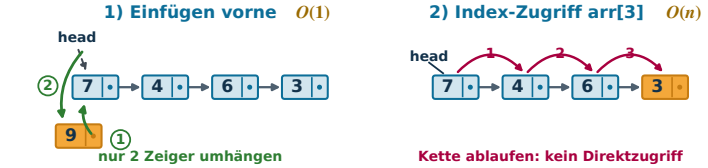
Elemente liegen als Knoten vor. Jeder Knoten speichert einen Wert und eine Referenz auf den nächsten Knoten.

- Vorteil** — Einfügen/Entfernen nach bekanntem Knoten ist $\mathcal{O}(1)$.
- Nachteil** — Suche nach Wert oder Position ist $\mathcal{O}(n)$.

Dummy-Knoten: zwei sortierte Listen mergen

```
class Node:  
    def __init__(self, val, next=None):  
        self.val = val  
        self.next = next  
  
def merge_sorted(a, b):  
    dummy = Node(0) # Start vor dem Ergebnis  
    tail = dummy # letztes Element im Ergebnis  
  
    while a is not None and b is not None:  
        if a.val <= b.val:  
            tail.next = a  
            a = a.next  
        else:  
            tail.next = b  
            b = b.next  
        tail = tail.next  
  
    tail.next = a if a is not None else b  
    return dummy.next
```

Operationen auf einer Linked List

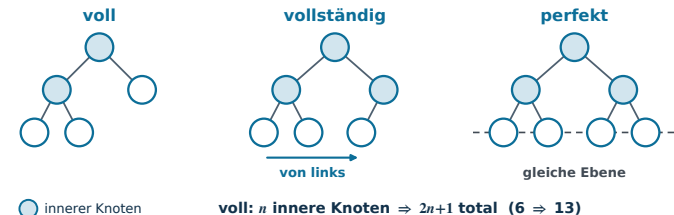


6.4. Binary (Search) Trees

Binärer Suchbaum (BST)

Für jeden Knoten gilt: Kleinere Werte stehen im linken, grössere Werte im rechten Teilbaum. Diese Regel bei jedem besuchten Knoten erneut anwenden.

Baumarten



Baumtiefe bei N Knoten

- Minimale Baumtiefe** — $\lceil \log_2(N+1) \rceil$
- Maximale Baumtiefe** — N

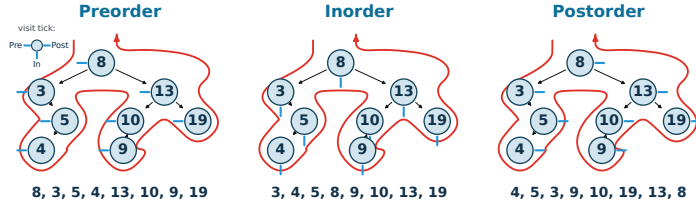
6.4.1. Komplexitäten

- Baum aufbauen (Worst Case)** — $\mathcal{O}(n^2)$ (balanciert: $\mathcal{O}(n \log n)$)
- Minimum/Maximum finden** — $\mathcal{O}(n)$ (balanciert: $\mathcal{O}(\log n)$)
- Einfügen** — $\mathcal{O}(n)$ (balanciert: $\mathcal{O}(\log n)$)
- Entfernen (beliebiges Element)** — $\mathcal{O}(n)$ (balanciert: $\mathcal{O}(\log n)$)

6.4.2. Traversierung: Baum in Liste (Pre-/In-/Postorder)

Baum in Liste umwandeln (rekursiv)

```
def preorder_list(node: Optional[Node]) -> list:  
    if node is None: # Basis: leer  
        return []  
    return [node.key] + preorder_list(node.left) # W->L->R  
        + preorder_list(node.right)  
  
def postorder_list(node: Optional[Node]) -> list:  
    if node is None: # Basis: leer  
        return []  
    return (postorder_list(node.left) # L->R->W  
            + postorder_list(node.right) + [node.key])  
  
def inorder_list(node: Optional[Node]) -> list:  
    if node is None: # Basis: leer  
        return []  
    return (inorder_list(node.left) # L->W->R  
            + [node.key] + inorder_list(node.right))
```



- Preorder** — Hauptreihenfolge, damit wird der Baum gebaut
- Postorder** — Nebenreihenfolge

6.4.3. Knoten hinzufügen (add)

Einfügen auf Papier

1. **Bei der Wurzel starten**
Neuen Wert mit dem aktuellen Knoten vergleichen.
2. **Pfad verfolgen**
Kleiner → links, grösser → rechts.
3. **Einfügen**
An der ersten freien Stelle einen neuen Knoten einsetzen.

```
add() iterativ                                avg.  $\mathcal{O}(\log n)$  • worst  $\mathcal{O}(n)$ 
def add(self, key: int) → bool:
    if self.root is None: # Leerer Baum
        self.root = Node(key)
        return True # Eingefügt
    n = self.root # Start bei Wurzel
    while n.key != key: # Bis Wert gefunden
        if key < n.key: # Kleiner → links
            if n.left is None: # Freie Stelle
                n.left = Node(key)
                return True # Eingefügt
            n = n.left # Weiter absteigen
        else: # Grösser → rechts
            if n.right is None: # Freie Stelle
                n.right = Node(key)
                return True # Eingefügt
            n = n.right # Weiter absteigen
    return False # Wert schon vorhanden
```

6.4.4. Knoten löschen (delete)

Löschen auf Papier

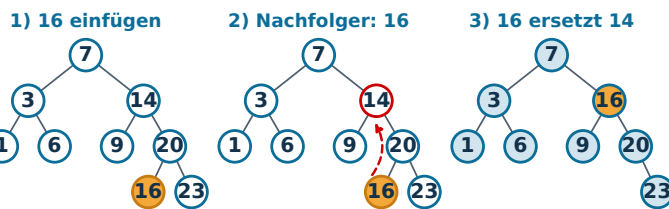
1. **Kein Kind**
Knoten entfernen.
2. **Ein Kind**
Das Kind übernimmt die Verbindung zum Elternknoten.
3. **Zwei Kinder**
Wert durch den symmetrischen Nachfolger ersetzen und dessen alten Knoten entfernen.
4. **Nachfolger finden**
Einmal rechts, danach so weit links wie möglich. Sein mögliches rechtes Kind übernimmt seine alte Verbindung.

```
deleteNode() rekursiv                        bal.  $\mathcal{O}(\log n)$  • worst  $\mathcal{O}(n)$ 
def deleteNode(root, key):
    if root is None: # Suchpfad endet
        return None
    if key < root.key: # Links weitersuchen
        root.left = deleteNode(root.left, key)
    elif key > root.key: # Rechts weitersuchen
        root.right = deleteNode(root.right, key)
    else: # Knoten gefunden
        if root.left is None: # Kein linkes Kind
            return root.right
        if root.right is None: # Kein rechtes Kind
            return root.left

        # 2 Kinder: symmetrischen Nachfolger übernehmen
        successor = findMin(root.right) # Kleinster rechts
        root.key = successor.key # Wert ersetzen
        root.right = deleteNode(root.right, successor.key)
    return root

def findMin(node):
    while node.left is not None: # So weit links wie möglich
        node = node.left
    return node # Kleinster Knoten
```

BST: Knoten mit zwei Kindern löschen



6.4.5. Knoten suchen (search)

```
search() iterativ                                avg.  $\mathcal{O}(\log n)$  • worst  $\mathcal{O}(n)$ 
def search(self, key: int) → Optional[Node]:
    n = self.root # Start bei Wurzel
    while n is not None and n.key != key: # Bis Fund/Ende
        if key < n.key: # Kleiner → links
            n = n.left
        else: # Grösser → rechts
            n = n.right
    return n # Knoten oder None
```

6.5. Heaps

6.5.1. Grundidee eines Binary Heaps

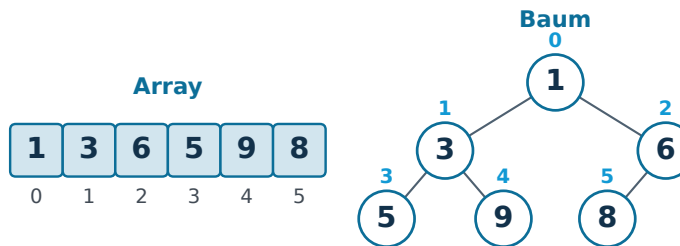
Binary Heap

Ein Binary Heap ist eine vollständige Binärstruktur, bei der gilt:

- **Min-Heap** — Elternknoten ist stets kleiner als seine Kinder.
- **Max-Heap** — Elternknoten ist stets grösser als seine Kinder.

Heaps werden in einer Liste gespeichert, sodass die Knotenebenen hintereinander notiert sind:

[1, 2, 3, 4, 5, 6] ⇒ Index: 0, 1, 2, ...



Zugriffsformeln

- $\text{parent} = (i - 1) // 2$
- $\text{left child} = 2i + 1$
- $\text{right child} = 2i + 2$
- $\text{Höhe} = \log_2(N + 1)$ mit $N = \text{len}(\text{list})$

6.5.2. Komplexitäten

- **Aufbauen mit siftUp** — $\mathcal{O}(n \log n)$ — Elemente nacheinander einfügen
- **Bottom-up-Heapify** — $\mathcal{O}(n)$ — ab letztem Elternknoten SiftDown
- **Maximum/Minimum holen** — $\mathcal{O}(1)$
- **Einfügen in Heap** — $\mathcal{O}(\log n)$
- **Entfernen (Top-Element)** — $\mathcal{O}(\log n)$

6.5.3. Heapify (Liste in Heap umwandeln)

Heapify: sift-up-Variante

```
def heapify(list):
    for i in range(len(list)): # Knoten einzeln einfügen
        while list[(i - 1) // 2] > list[i] and i > 0:
            swap(list, (i - 1) // 2, i) # Mit Eltern tauschen
            i = (i - 1) // 2 # Beim Elternknoten weiter
    return
```

Heapify: bottom-up-Variante

```
def heapify(list):
    n = len(list)
    # Eltern: unten rechts bis zur Wurzel
    for i in range(n // 2 - 1, -1, -1):
        shift_down(list, i, n) # Teilbaum reparieren
    return
```

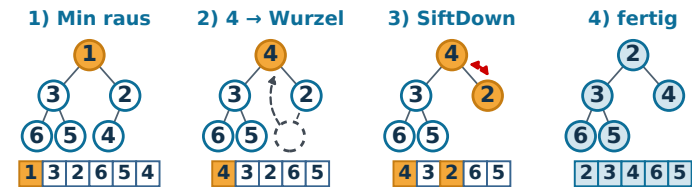
6.5.4. SiftDown (Element nach unten schieben)

Beim Entfernen der Wurzel (z. B. Max-Element bei Max-Heap) wird das letzte Element nach oben geschoben und mit seinen Kindern verglichen. Dabei wird immer mit dem grösseren (bzw. kleineren bei Min-Heap) Kind getauscht.

SiftDown nach Entfernen

```
def SiftDown(a, i, m):
    while 2*i + 1 < m: # Solange ein Kind existiert
        j = 2*i + 1 # Zuerst linkes Kind wählen
        if j + 1 < m and a[j] < a[j + 1]:
            j += 1 # Rechtes Kind ist grösser
        if a[i] >= a[j]: # Eltern >= grösstes Kind
            break
        a[i], a[j] = a[j], a[i] # Nach unten tauschen
        i = j # Eine Ebene tiefer weiter
```

Min-Heap: Wurzel entfernen



6.5.5. Sortieren mit Heaps (Heap Sort)

Heap Sort mit SiftDown

```
def SortHeap(a):
    heapify(a) # Max-Heap aufbauen
    for i in reversed(range(len(a))):
        swap(a[0], a[i]) # Grössten Wert ans Ende
        SiftDown(a, 0, i) # Restlichen Heap reparieren
    return
```

6.5.6. Elemente einfügen und löschen

Einfügen auf Papier

1. **Platz wählen**
Neuen Knoten unten an der ersten freien Stelle von links einsetzen.
2. **Nach oben tauschen**
Min-Heap: kleinere Werte nach oben; Max-Heap: grössere Werte nach oben. Mit dem Elternknoten tauschen, solange die Reihenfolge falsch ist.

Top entfernen auf Papier

1. **Wurzel ersetzen**
Wurzelwert entfernen. Wert unten ganz rechts an die Wurzel setzen und den unteren Knoten löschen.
2. **Nach unten tauschen**
Im Min-Heap das kleinere, im Max-Heap das grössere Kind wählen. Mit diesem Kind tauschen, solange die Reihenfolge falsch ist.

Heap aufbauen und kontrollieren

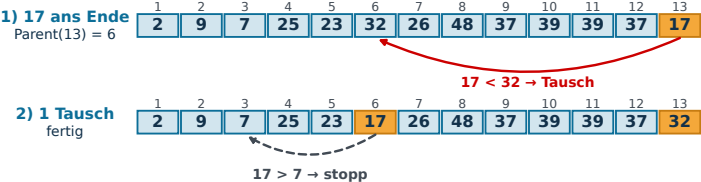
1. Unten beginnen

Bei den untersten Eltern rechts starten. Jeden Wert wie beim Entfernen nach unten tauschen. Danach Ebene für Ebene bis zur Wurzel weitergehen.

2. Kontrolle

Unten sind die Knoten von links gefüllt, ohne Lücke. Jeder Elternwert steht in der richtigen Reihenfolge zu seinen Kindern.

Min-Heap: Einfügen und SiftUp



Indexierung in der Grafik

Die Grafik zählt Positionen ab 1; die Python-Formeln oben verwenden Indizes ab 0.

```
removeMax()  
def removeMax(heap):  
    if len(heap) == 0:  
        return None  
    max_val = heap[0] # Wurzel sichern  
    heap[0] = heap[-1] # Letzten Knoten nach oben  
    heap.pop() # Alten letzten Knoten löschen  
    SiftDown(heap, 0, len(heap)) # Nach unten reparieren  
    return max_val
```

6.6. Hash-Tabellen

6.6.1. Hash: Grundidee Hash-Tabelle

Eine Hash-Tabelle speichert ungeordnete Sammlungen **einzigartiger Elemente** (z. B. set oder dict in Python).

Hashing: Einer Eingabe wird mittels **Hash-Funktion** ein Integer zugewiesen. Die Platzierung im Array der Länge M erfolgt am Index:

$$\text{index} = \text{hash}(x) \% M \quad (\text{math: mod})$$

6.6.2. Laufzeiten

- Suche, Einfügen, Entfernen (Erwartungswert) — $\mathcal{O}(1)$
- Suche, Einfügen, Entfernen (Worst Case) — $\mathcal{O}(n)$

6.6.3. Kollisionsbehandlung

Treffen mehrere Elemente auf denselben Index, gibt es zwei Lösungsansätze:

- Chaining** — Am Index wird der Head-Knoten einer **Linked List** gespeichert, die alle kollidierenden Elemente aufnimmt. **6.3**
Vorteil: Ein oft wiederholter Hash blockiert nicht die restliche Tabelle.
- Probing** — Ist der Index besetzt, wird zirkulär der nächste freie Index (**Wrap-around**) gesucht.
Vorteil: Alle Daten bleiben Cache-freundlich im Array.
Nachteil: Löschungen erfordern **Tombstoning** (spezielle Literale als Platzhalter) oder komplexes Rückwärts-Verschieben.

Lineares Probing

Lineares Probing

26 → Idx 6

2 Hops rechts

13, 3 → Idx 2, 3

0 Hops

32 → Idx 0

1 Hop + Wrap

Chaining mit Buckets

```
def our_hash(word):  
    return ord(word[0]) # einfache Hash-Fkt.  
  
def create_hash_table(l, M):  
    res = [[] for _ in range(M)] # M leere Buckets  
    for x in l:  
        idx = our_hash(x) % M # % statt mod!  
        if x not in res[idx]: # keine Duplikate  
            res[idx].append(x) # chaining: anhängen  
    return res
```

6.7. Quadrees

6.7.1. Quadtree: Idee & Struktur Quadtree & Struktur

Quadtrees sind **volle Quaternärbäume** (0 oder 4 Kinder) zur effizienten Verarbeitung **räumlicher 2D-Daten** (z. B. Rechteck-Abfragen).

- l, u — untere linke Ecke / obere rechte Ecke u
- m — Maximalkapazität
- p — Liste von Punkten
- $c_0..c_3$ — 4 Kinder-Quadrees

Quadtree-Knoten

```
class QuadTree:  
    def __init__(self, l, u, m):  
        self.l, self.u = l, u # untere/obere Ecke  
        self.m = m # Kapazität  
        self.p = [] # Punkte im Knoten  
        self.c = [None]*4 # Kinder c0..c3
```

6.7.2. Einfügen & Split

Ablauf

1. Einfügen

Ein Punkt wird zu p hinzugefügt.

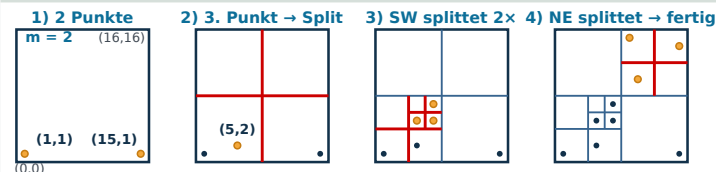
2. Split

Überschreitet die Anzahl der Punkte die Kapazität m , wird der Knoten in **vier Viertel** partitioniert.

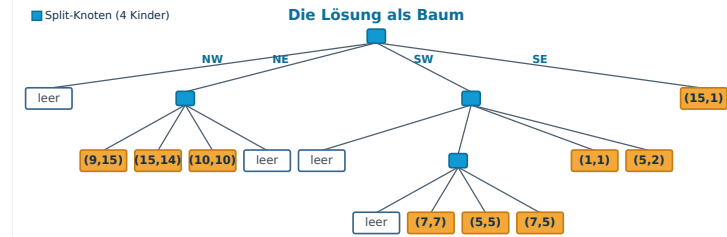
3. Aufteilen

Die Punkte aus p werden auf die Kinder aufgeteilt und das eigene p geleert.

Konstruktion



Prozessierender Baum



6.7.3. Rechtecks-Abfrage (Range Query)

- Laufzeit** — Die Suche wächst logarithmisch mit der Anzahl der Punkte: $\mathcal{O}(\log n)$.
- Pruning** — Schneidet das Such-Rechteck einen Quadranten nicht, wird dieser **Zweig komplett ignoriert**.

Quadtree: Punkte in Rechteck suchen (Range Query)

```
def query(node, q_l, q_u): # q_l/q_u: Grenzen →Punktliste  
    if not overlaps(node, q_l, q_u): # Pruning!  
        return [] # Zweig ignorieren  
    hits = [pt for pt in node.p # lokale Punkte  
            if inside(pt, q_l, q_u)]  
    for child in node.c: # rekursiv in Kinder  
        if child is not None:  
            hits += query(child, q_l, q_u)  
    return hits
```

6.7.4. Wann lohnt sich ein Quadtree?

Die **Konstruktion** des Baumes ist relativ aufwendig; der Einsatz lohnt sich daher erst bei **höherfrequentierten Abfragen**.

7. Numpy

7.1. Array-Grundlagen

7.1.1. Array erzeugen / Konstruktoren

```
np.array([1, 2, 3]) # Array aus Python-Liste erstellen  
np.asarray(x) # kein Copy falls x schon Array  
np.copy(x) # tiefe Kopie: unabh. vom Original  
np.zeros((3, 3)) # 3x3-Matrix, alle Werte = 0.0  
np.ones((2, 4)) # 2x4-Matrix, alle Werte = 1.0  
np.full((2, 2), fill_value=9) # 2x2-Matrix, alle Werte = 9  
# gr1xgr2-Array, alle Werte = None (dtype=object)  
vk = np.full((gr1, gr2), fill_value=None)  
np.eye(4) # Einheitsmatrix: 1 auf Hauptdiag.  
np.arange(1, 10, 2) # [1,3,5,7,9] von 1 bis <10, Schr. 2  
np.linspace(0, 1, 100) # 100 gleichm. Werte von 0 bis 1  
np.logspace(0, 3, 4) # [1,10,100,1000]: 10^0 bis 10^3  
np.empty((2, 2)) # nicht init. -zuf. Speicherinhalt
```

7.1.2. Array-Informationen

```
a.shape # Tupel der Dimensionen (z.B. (3,4) = 3x4)  
# Anzahl Dimensionen (2 für Matrix, 1 für Vektor)  
a.ndim  
a.size # Gesamtanzahl Elemente (Produkt der Dimensionen)  
a.dtype # Datentyp der Elemente (z.B. int32, float64)  
a.itemsize # Bytes pro Element (z.B. 4 für int32)  
a.nbytes # Gesamtspeicher in Bytes (= size * itemsize)  
a.T # Transponierte Matrix  
a.strides # Schrittgrösse pro Achse in Bytes  
a.flags # Speicherlayout-Flags (C_CONTIGUOUS, etc.)  
a.base # Basisdaten: None=Owner, sonst Quelle des Views
```

7.1.3. Typumwandlung

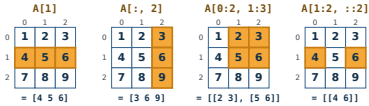
```
a.astype(np.float32) # in float32 umwandeln (kopiert)  
# kein Copy falls dtype schon passt  
a.astype("int32", copy=False)  
a.tolist() # zurück zur Python-Liste
```

7.2. Zugriff und Shape-Operationen

7.2.1. Indexing, Slicing, Maskierung

```
a[1], a[1, :], a[:, 0] # Element / ganze Zeile / ganze Spalte
a[1:4], a[:,2]         # Slicing / jedes 2. Element
a[a > 3]               # boolescher Indexzugriff (Maske)
# bedingte Ersetzung: a wenn >0, sonst 0
np.where(a > 0, a, 0)
np.nonzero(a)           # Indizes aller Nicht-Null-Elemente
np.take(a, [0, 2])      # Elemente per Index sammeln
# Elemente per Index setzen (in-place)
np.put(a, [1, 3], [9, 10])
np.compress(a > 2, a)   # Elemente filtern per boolescher Maske
```

2D-Slicing



Boolean Mask



7.2.2. Broadcasting & Form ändern

```
a + b                # Shape automatisch erweitern
# resultierenden Shape bestimmen
np.broadcast_shapes(a.shape, b.shape)
np.broadcast_to(a, (3, 4)) # manuelles Broadcasting (View)
np.expand_dims(a, axis=0)  # neue Achse einfügen
np.squeeze(a)             # Achsen der Größe 1 entfernen
np.reshape(a, (2, -1))    # Form ändern (-1 = auto)
np.tile(a, (2, 3))        # Muster 2x vertikal, 3x horiz.
```

7.2.3. Stacking & Splitting (Zusammenfügen / Aufteilen)

```
# Kombinieren von Arrays
np.concatenate((a, b), axis=0) # entlang bestehender Achse
np.stack([a, b], axis=0)       # neue Achse einfügen
np.vstack([a, b])              # vertikal stapeln (Zeilen)
np.hstack([a, b])              # horizontal stapeln (Spalten)
np.dstack([a, b])              # tiefen-stapeln (3. Achse)
# Aufteilen von Arrays
np.split(a, 3)                 # in 3 gleiche Teile aufteilen
np.array_split(a, 3)           # in 3 Teile (ungleiche OK)
np.vsplit(a, 2)                # vertikal in 2 Hälften teilen
np.hsplit(a, 2)                # horizontal in 2 Hälften teilen
```

7.3. Berechnen und Zusammenfassen

7.3.1. Elementweise Mathematik

```
np.add(a, b), np.subtract(a, b) # Addition, Subtraktion
np.multiply(a, b), np.divide(a, b) # Multiplikation, Division
np.mod(a, b), np.remainder(a, b) # Modulo / Rest
np.exp(a), np.log(a), np.log10(a) # e^a, ln(a), log10(a)
np.sqrt(a), np.square(a)         # Wurzel, Quadrat (a^2)
np.clip(a, min, max)             # Werte auf [min, max] begrenzen
np.sign(a), np.abs(a)            # Vorzeichen (-1/0/1), Betrag
# Aufrunden, Abrunden, Runden
np.ceil(a), np.floor(a), np.round(a)
```

7.3.2. Aggregate und Statistik

```
np.sum(a), np.prod(a)           # Summe, Produkt aller Elemente
np.mean(a), np.median(a)        # arithm. Mittelwert, Median
np.std(a), np.var(a)            # Standardabweichung, Varianz
np.min(a), np.max(a)            # kleinster, grösster Wert
np.argmin(a), np.argmax(a)      # Index von Min bzw. Max
np.ptp(a)                       # Peak-to-Peak: max(a) - min(a)
np.percentile(a, 75)            # 75. Perzentil
np.histogram(a, bins=10)        # Histogramm berechnen
```

8. Pandas

8.1. DataFrame-Grundlagen

8.1.1. DataFrame erzeugen

```
import pandas as pd
df = pd.DataFrame() #leerer DataFrame
df = pd.read_csv("Name.csv") #DataFrame aus CSV laden
```

8.1.2. Zeilen/Spalten auswählen (loc / iloc)

```
data["Month"] #einzelne Spalte
data[["Name", "City"]] #mehrere Spalten
data["Name"][2] #gibt 'Carlos' zurück
data.iloc[1] #Zeile mit Index 1
data[1:3] #Zeilen mit Index 1 und 2
data.loc[1:3, "M":"C"] #Zeilen 1-3, Spalten M bis C
data.iloc[1:3, 0:2] #wie loc, aber indexbasiert
```

8.1.3. Zeilen filtern (Bedingung / boolean mask)

```
data[data["Age"] > 30] #alle Personen über 30
data["Name"][data["Age"] > 30] #nur Namen dieser Personen
```

8.2. Daten verändern und bereinigen

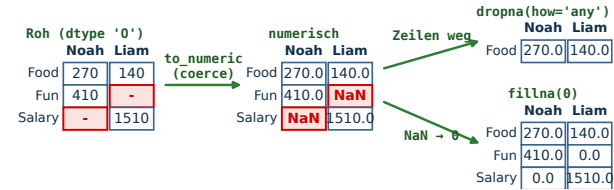
8.2.1. Index & Spalten umbenennen

```
data.set_index("Row") #setzt 'Row' als Index
data.rename(columns={"City": "Location"}) #Spalte umbenennen
#jede Spalte umbenennen
data.columns = ["Monat", "Name", "Stadt", ...]
```

8.2.2. Löschen & fehlende Werte (NaN)

```
data.drop(columns=["Age"]) #löscht Spalte 'Age'
data.drop(data.index[0]) #löscht erste Zeile
#löscht Zeilen mit NaN
data.dropna(axis=0, how="any") #wenn mind. 1 NaN in der Zeile
data.dropna(axis=0, how="all") #wenn alle NaN in der Zeile
data.dropna(axis=1, how="any") #löscht Spalten mit mind. 1 NaN
data.fillna(...) #ersetzt NaN durch gewünschten Wert
```

Typen und fehlende Werte bereinigen



8.3. Auswerten

8.3.1. Hilfsfunktionen für DataFrames

```
data["Age"].sum() #summiert alle Werte der Spalte 'Age'
data["Age"].max() #maximaler Wert
```

9. ML

9.1. Klassifikation: direkt zur Codeaufgabe

Aufgabe erkannt

Die Daten besitzen bekannte Labels, das Ziel ist eine von mehreren Klassen und die verlangte Bewertung ist Accuracy: Das ist eine überwachte Klassifikationsaufgabe.

Was verlangt die Aufgabe? → Direkt zu

- Bestimmter Classifier — 9.3 Modellblöcke.
- Modell frei; beste Accuracy — 9.4 Modellvergleich.
- Hyperparameter optimieren — 9.6 GridSearchCV.
- Accuracy zu niedrig — 9.5 Konkrete Änderungen.
- Begriffe oder Formeln erklären — 9.7 Theorie und Referenz.
- Stetigen Wert vorhersagen — 9.7.4 Regression.

9.2. Standard: Grundmodell trainieren

Ziel: CSV-Daten in X/y aufteilen, ein Grundmodell trainieren und seine Test-Accuracy bestimmen.

Merksatz

Daten aufteilen, Modell erstellen, fit, predict, bewerten.

Grundmodell: Pflichtschema

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

ANPASSEN: Datei und Name der Labelspalte

```
df = pd.read_csv("data.csv")
X = df.drop(columns=["target"])
y = df["target"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2, # 20% bleiben ungesehen zum Testen
    random_state=0 # gleicher Split bei jedem Lauf
)
```

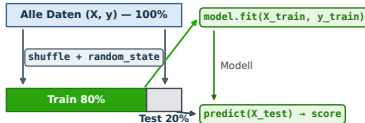
```
model = RandomForestClassifier(random_state=0)
model.fit(X_train, y_train)
```

```
predictions = model.predict(X_test)
test_accuracy = accuracy_score(y_test, predictions)
```

Neue Daten vorhersagen

```
X_new = pd.read_csv("new_data.csv")
predictions = model.predict(X_new)
```

Train-Test-Split



9.3. Vorgegebenes Modell einsetzen

Ziel: Wenn die Aufgabe ein Modell vorgibt, genau diesen model-Block wählen; Split, fit, predict und Bewertung bleiben wie in 9.2.

DecisionTreeClassifier

```
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(
    max_depth=8, # begrenzt starkes Overfitting
    min_samples_leaf=2, # robustere Blätter
    random_state=0
)
```

RandomForestClassifier

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(
    n_estimators=200, # Anzahl Bäume
    max_depth=None, # kleiner = stärkere Begrenzung
    random_state=0
)
```

LogisticRegression mit Skalierung

```
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

model = make_pipeline(
    StandardScaler(), # Skalierung gehört ins Modell
    LogisticRegression(max_iter=1000)
)
```

KNeighborsClassifier mit Skalierung

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
model = make_pipeline(
    StandardScaler(),          # Distanzen vergleichbar machen
    KNeighborsClassifier(n_neighbors=5) # Anzahl Nachbarn
)
```

MLPClassifier mit Skalierung

```
from sklearn.neural_network import MLPClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
model = make_pipeline(
    StandardScaler(),          # wichtig für stabiles Training
    MLPClassifier(
        hidden_layer_sizes=(100,), # robuster Standard: eine Schicht
        max_iter=1000,             # erhöhen, falls keine Konvergenz
        alpha=0.0001,              # grösser = mehr Regularisierung
        random_state=0
    )
)
```

9.4. Alternative: freie Modellwahl

Ziel: Split aus 9.2 übernehmen, mehrere Modelle gleich trainieren und das Modell mit der höchsten Test-Accuracy wählen.

Modellvergleich: 1. Importe

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
```

Modellvergleich: 2. Kandidaten-Dict

```
# Tree/Forest brauchen keine Skalierung.
# Bei LogReg/KNN/MLP skaliert die Pipeline automatisch.
models = {
    "Tree": DecisionTreeClassifier(
        max_depth=8, min_samples_leaf=2, random_state=0),
    "Forest": RandomForestClassifier(
        n_estimators=200, random_state=0),
    "LogReg": make_pipeline(
        StandardScaler(), LogisticRegression(max_iter=1000)),
    "KNN": make_pipeline(
        StandardScaler(), KNeighborsClassifier(n_neighbors=5)),
    "MLP": make_pipeline(
        StandardScaler(), MLPClassifier(
            hidden_layer_sizes=(100,), max_iter=1000,
            random_state=0))
}
```

Modellvergleich: 3. Benchmark-Schleife

```
from sklearn.metrics import accuracy_score

scores = {} # Name → Test-Accuracy

for name, candidate in models.items(): # Alle Modelle durchgehen
    candidate.fit(X_train, y_train) # Modell trainieren
    predictions = candidate.predict(X_test) # Testdaten vorhersagen
    scores[name] = accuracy_score(y_test, predictions) # Accuracy speichern

best_name = max(scores, key=scores.get) # Key (Name) mit max. Wert
best_model = models[best_name] # Gewinner-Modell holen
best_accuracy = scores[best_name] # Beste Accuracy
```

9.5. Accuracy-Problem erkennen und verbessern

```
Train- und Test-Accuracy berechnen
train_predictions = model.predict(X_train)
train_accuracy = accuracy_score(y_train, train_predictions)

test_predictions = model.predict(X_test)
test_accuracy = accuracy_score(y_test, test_predictions)
```

1. Problem an Train- und Test-Accuracy erkennen

Beobachtung	Problem	Bedeutung
Training hoch, Test deutlich tiefer	Overfitting	Modell merkt sich Trainingsdaten zu stark.
Training und Test beide tief	Underfitting	Modell erkennt das Muster noch nicht gut genug.
Training und Test beide hoch	kein Problem	Modell generalisiert auf ungesehene Daten.
Ergebnis schwankt stark	instabil	Datensatz klein oder Modell stark vom Split abhängig.

2. Beim verwendeten Modell den passenden Parameter ändern

Modell	Problem	Änderung
Decision Tree	Overfitting	max_depth kleiner oder min_samples_leaf grösser
Decision Tree	Underfitting	max_depth grösser
Random Forest	instabil / zu schwach	n_estimators erhöhen
Logistic Regression	konvergiert nicht	skalieren, max_iter erhöhen
Logistic Regression	Underfitting	C erhöhen
KNN	Overfitting	n_neighbors erhöhen
KNN	Underfitting	n_neighbors verkleinern
MLP	konvergiert nicht	skalieren, max_iter erhöhen
MLP	Overfitting	alpha erhöhen oder Netz verkleinern
MLP	Underfitting	Netz vergrössern

9.6. Alternative: Hyperparameter mit GridSearchCV suchen

Nur verwenden, wenn die Aufgabe ausdrücklich GridSearchCV oder eine automatische Parametersuche verlangt. Das Beispiel testet passende Werte für einen Decision Tree.

Merksatz

param_grid enthält die Werte, die ausprobiert werden. search wird wie das Modell benutzt: immer zuerst fit, dann predict.

GridSearchCV: Pflichtschema

```
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(random_state=0) # Basis-Modell

param_grid = { # Parameter, die ausprobiert werden
    "max_depth": [None, 4, 8], # Verschiedene Baumtiefen
    "min_samples_leaf": [1, 5] # Verschiedene Blattgrössen
}

search = GridSearchCV( # Suche nach besten Parametern konfigurieren
    estimator=model,
    param_grid=param_grid,
    cv=5 # 5 Test-Durchläufe (Cross-Validation)
)

search.fit(X, y) # Alle Kombis testen & beste finden
predictions = search.predict(X_new) # Vorhersage mit besten Parametern
```

Siehe Theorie zu Grid Search und Cross-Validation (→ 9.7.2)

9.7. Theorie und Modellreferenz

9.7.1. Regression vs. Klassifikation

- **Regression** — sagt einen stetigen Zahlenwert voraus, z. B. einen Preis.
- **Klassifikation** — ordnet einen Datenpunkt einer festen Klasse zu, z. B. Spam oder Nicht-Spam.
- **Supervised Learning** — Training mit Eingaben X und bekannten Zielwerten oder Labels y.
- **Faustregel** — Zahl mit Einheit oder Skala ⇒ Regression; endliche feste Kategorien ⇒ Klassifikation.

Regression oder Klassifikation?

Szenario	Typ	Szenario	Typ
Temperatur morgen	Reg.	Diabetes ja/nein	Klass.
Hauspreis	Reg.	Betrug ja/nein	Klass.
Prüfungsnote	Reg.	Wahrscheinl. %	Reg.

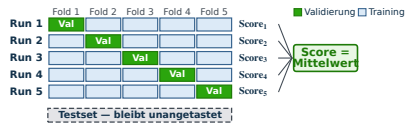
9.7.2. Wichtige Parameter

- **max_iter** — Maximale Anzahl Trainingsiterationen; wichtig für LogisticRegression, MLPClassifier und MLPRegressor.
- **hidden_layer_sizes** — Anzahl Neuronen pro versteckter Schicht bei MLP-Modellen.
- **random_state** — Fixiert Zufallsprozesse für reproduzierbare Ergebnisse; ist kein Tuning-Parameter für bessere Accuracy.
- **test_size** — Anteil der Daten im Testset; üblich sind 0.1 bis 0.3.
- **cv** — Anzahl Folds bei GridSearchCV; 5 ist ein üblicher Standard.

Grid Search und K-Fold-Cross-Validation

- **Grid Search** — probiert vorgegebene Hyperparameter-Kombinationen und vergleicht ihre Scores.
- **K-Fold** — trainiert das Modell mehrfach; jeder Fold dient einmal als Validierungsmenge. Der Mittelwert schätzt die Modellgüte stabiler als ein einzelner Split.

K-Fold-Cross-Validation



9.7.3. Klassifikationsmodelle

Logistic Regression

- **Idee** — lineare Entscheidungsgrenze; liefert Klassenwahrscheinlichkeiten.
- **Modell** — Die Wahrscheinlichkeit der positiven Klasse ist $y = \sigma(w_0 + w_1x_1 + \dots + w_nx_n)$.
- **Schwelle** — Ausgabe ist eine Wahrscheinlichkeit; die Schwelle 0.5 trennt die Klassen.
- **Unsicher** — An der Entscheidungsgrenze ist die Vorhersage ≈ 0.5 (maximal unsicher); liegt sie fast überall bei 0.5, wurde nichts gelernt.
- **Stärken** — schnell und interpretierbar.
- **Schwächen** — schwierig bei stark nicht-linearen Mustern; Skalierung ist sinnvoll.
- **Parameter** — C, class_weight, max_iter.

Decision Tree Classifier

- **Idee** — zerlegt den Merkmalsraum durch Entscheidungen und sagt in jedem Blatt eine Klasse voraus.
- **Stärken** — interpretierbar; benötigt keine Skalierung.
- **Schwächen** — anfällig für Overfitting.
- **Parameter** — max_depth, min_samples_split, min_samples_leaf.

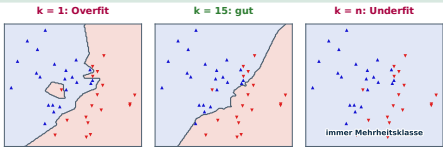
Random Forest Classifier

- **Idee** — kombiniert viele zufällig variierte Decision Trees und lässt sie abstimmen.
- **Stärken** — oft robuster als ein einzelner Tree; benötigt keine Skalierung.
- **Mehr Bäume** — ein grösseres n_estimators macht robuster, führt aber **nicht** zu Overfitting — nur mehr Rechenzeit.
- **Schwächen** — weniger leicht interpretierbar und rechenintensiver.
- **Parameter** — n_estimators, max_depth, min_samples_leaf.

KNN

- **Idee** — Klasse durch Mehrheitsentscheid der k nächsten Trainingspunkte.
- **Kleines k** — flexibel, aber anfällig für Overfitting.
- **Grosses k** — glatter, kann aber underfitten.
- **Wichtig** — Features skalieren; n_neighbors und weights tunen.
- **Lazy Learning** — kein echtes Training — rechnet erst bei predict alle Distanzen (langsam bei grossen Daten, unzuverlässig in vielen Dimensionen: Fluch der Dimensionalität).
- **Faustregeln** — $k = n$ ergibt immer die Mehrheitsklasse; ungerades k vermeidet Gleichstände.

KNN: Einfluss von k



MLPClassifier

- **Idee** — neuronales Netz für nicht-lineare Klassifikationsaufgaben. (→ 10.1)
- **Stärken** — flexibel für komplexe Muster.
- **Schwächen** — mehr Tuning und Rechenzeit; Features skalieren.
- **Parameter** — hidden_layer_sizes, activation, alpha, max_iter.

Skalierung: grosse Werte nicht dominieren lassen

- **Ja** — KNN, MLP, K-Means, PCA — distanz- bzw. gradientenbasiert, reagieren auf die Feature-Skala.
- **Nein** — Decision Tree, Random Forest — arbeiten mit Schwellen-Splits und sind skalenunabhängig.

9.7.4. Regressionsmodelle

Ziel des Codes: Ein Regressionsmodell trainieren und für X_test stetige Zahlenwerte vorhersagen.

Linear Regression

- **Idee** — Vorhersage durch eine lineare Kombination der Eingabefeatures: $y = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$.
- **Stärken** — schnell, interpretierbar und theoretisch gut verstanden.
- **Schwächen** — empfindlich gegenüber Ausreissern; kann komplexe Muster nicht abbilden.
- **Parameter** — fit_intercept steuert den Bias-Term; n_jobs die verwendeten CPU-Kerne.
- **Import** — from sklearn.linear_model import LinearRegression

LinearRegression

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Decision Tree Regressor

- **Idee** — modelliert nicht-lineare Beziehungen durch Entscheidungen an Verzweigungsknoten.
- **Prinzip** — zerlegt die Eingabemenge in Zonen mit möglichst homogener Zielvariable.
- **Import** — from sklearn.tree import DecisionTreeRegressor

Neural Network Regressor

- **Idee** — neuronales Netz mit Regressionsziel; nicht-linear und flexibel.
- **Tuning** — hidden_layer_sizes, max_iter, activation.
- **Import** — from sklearn.neural_network import MLPRegressor

9.8. Modellbewertung und Loss

9.8.1. Accuracy und Balanced Accuracy

Binäre Confusion Matrix

real / vorhergesagt	Positiv	Negativ
Positiv	True Pos. (TP)	False Neg. (FN)
Negativ	False Pos. (FP)	True Neg. (TN)

Accuracy — Anteil korrekter Klassifikationen:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Balanced Accuracy — Mittelwert der Trefferquoten (binär):

$$\text{Balanced Accuracy} = \frac{1}{2} (\text{TPR} + \text{TNR}), \quad \text{TPR} = \frac{TP}{TP + FN}, \quad \text{TNR} = \frac{TN}{TN + FP}$$

Mehrere Klassen

Bei mehreren Klassen ist Balanced Accuracy der Mittelwert des Recalls aller Klassen. Die Confusion Matrix besitzt dann eine Zeile und Spalte pro Klasse.

9.8.2. 0-1-Loss (Error Rate)

Die **0-1-Loss-Funktion** ist 0 bei einer korrekten und 1 bei einer falschen Vorhersage:

$$L(y, \hat{y}) = \begin{cases} 0, & y = \hat{y}, \\ 1, & y \neq \hat{y}. \end{cases}$$

Das empirische Risiko ist der Mittelwert $\frac{1}{n} \sum_{i=1}^n L(y_i, \hat{y}_i)$. Bei binärer Klassifikation:

$$\text{Error Rate} = \frac{FP + FN}{TP + TN + FP + FN}, \quad \text{Accuracy} = 1 - \text{Error Rate}.$$

9.8.3. Gini-Index (Split-Kriterium)

Intuition: Der Gini-Index misst, wie gemischt die Klassen in einem Knoten sind: 0 bedeutet rein (nur eine Klasse), ein grosser Wert bedeutet stark gemischt. Ein Tree bevorzugt Splits mit kleinem Gini-Index, weil ihre Teilmengen klarer einer Klasse zuzuordnen sind.

Die Impurity einer Partition $Part_m$ ist:

$$G(Part_m) = \sum_k \hat{p}_{m,k} (1 - \hat{p}_{m,k}).$$

Für einen Split in $Part_1$ und $Part_2$:

$$G_{\text{split}} = \frac{|Part_1|}{|Part_m|} G(Part_1) + \frac{|Part_2|}{|Part_m|} G(Part_2).$$

Ein Decision Tree wählt den Split mit kleinem Gini-Index.



9.8.4. R2-Score und Mean Squared Error

Der R^2 -Score misst die Qualität einer Regressionsvorhersage:

$$R^2(D, f) = 1 - \frac{MSE(D, f)}{MSE(D, f_0)}$$

- **Lesen** — $R^2 = 1$: perfekt; 0: nicht besser als immer den Durchschnitt vorhersagen; < 0 : schlechter.
- f_0 — konstante Durchschnittsvorhersage.
- **MSE** — mittlere quadratische Abweichung: $MSE(D, f) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$.

- **MSE vs. MAE** — MSE quadriert Fehler, deshalb dominieren Ausreisser (Standardfall). $MAE = \frac{1}{n} \sum_i |y_i - \hat{y}_i|$ gewichtet linear und ist robust gegen Ausreisser.
- **Skalierung** — MSE hängt von der Skalierung der Zielwerte ab, R^2 nicht.

9.9. Kategorische Daten encodieren

Ziel des Codes: Kategoriewerte in numerische Spalten umwandeln, bevor sie als Features verwendet werden.

Maschinelles Lernen erfordert in der Regel numerische Eingabedaten. Nicht-numerische kategorische Daten wie „Farbe“ oder „Beruf“ müssen zuerst numerisch encodiert werden.

Encoding-Verfahren im Vergleich

Verfahren	Pro	Con
Ordinal	Einfach umsetzbar. Ermöglicht die Abbildung geordneter Kategorien.	Kann eine falsche Nähe zwischen Kategorien suggerieren und Modellinterpretationen verzerren.
Mean	Nutzt den Mittelwert der Zielvariable pro Kategorie und kann nützliche Information übertragen.	Gefahr von Data Leakage ; erhöhtes Overfitting-Risiko.
One-hot	Separate Spalte pro Kategorie; geeignet für nicht-ordinale Kategorien.	Bei vielen Kategorien entstehen sehr viele Spalten.

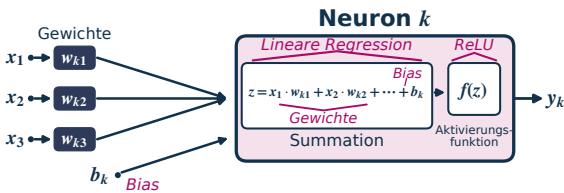
One-Hot-Encoding mit pandas

```
import pandas as pd
```

```
pd.get_dummies(a)
```

10. Neuronale Netzwerke

Neuronale Netzwerke (Neural Networks, NNs) bestehen aus vielen kleinen Funktionseinheiten, den sogenannten **Neuronen**. Diese eignen sich besonders gut zur Erkennung komplexer Muster in strukturierten Daten, Bildern oder Text.



10.1. MLP (Standard Neural Network)

10.1.1. Neuron, Gewichte und Bias

Ein Neuron berechnet:

$$y_k = f(x_1 \cdot w_{k1} + x_2 \cdot w_{k2} + \dots + x_n \cdot w_{kn} + b_k)$$

- x_i — Eingabewerte
- w_{ki} — Gewichtung: Wie stark eine Eingabe zählt
- b_k — Bias: verschiebt die Reaktionsschwelle
- f — Aktivierungsfunktion: formt das berechnete Signal (siehe unten)
- **Ablauf** — Gewichten → Bias addieren → Aktivierung anwenden
- **Training** — Gewichtungen w_{ki} und Bias b_k werden im Training angepasst

10.1.2. Aktivierungsfunktionen

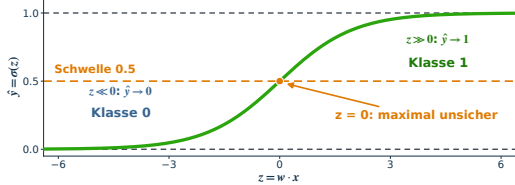
ReLU und Sigmoid

- **ReLU** — Negative Werte → 0, positive bleiben; oft in versteckten Schichten
- **Sigmoid** — $\sigma(x) = \frac{1}{1+e^{-x}}$ — Ausgabe bei binärer Klassifikation

Warum nichtlinear?

Nur lineare Aktivierung ($f(x) = x$) lässt ein tiefes Netz zu einer einzigen linearen Schicht zusammenfallen. Erst Nichtlinearität (ReLU, Sigmoid) macht zusätzliche Schichten ausdrucksstark.

Sigmoid: interner Wert → Wahrscheinlichkeit



10.2. Softmax (Ausgabe für mehrere Klassen)

- Idee** — Wandelt Ausgaben in Wahrscheinlichkeiten um:

$$p_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)} \quad \text{mit} \quad \sum_i p_i = 1$$

- Einsatz** — Wird bei Mehrklassenklassifikation als Ausgabeschicht genutzt
- Lesen** — Klassen konkurrieren; alle Wahrscheinlichkeiten zusammen = 1.

10.3. CNN (Convolutional Neural Network für Bilder)

10.3.1. Convolution und Pooling

Ein Neuron in einem Convolutional Neural Network (CNN) entsteht durch folgende Schritte:

- Convolutional Filter** — Kleiner Filter (z. B. 2×2) wird auf Bildbereiche angewendet. Ergebnis wird durch eine Aktivierungsfunktion verarbeitet (z. B. ReLU)
- Pooling Layer** — Kleine Pooling-Matrix (z. B. 2×2) mit Aggregationsfunktion, typischerweise **MAX**

1. Filter anwenden

Anwenden des Filters auf die Eingabematrix

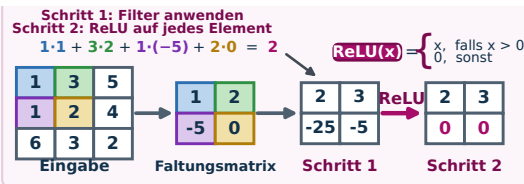
2. ReLU anwenden

Anwenden der ReLU-Funktion auf jedes Element des Ergebnisses (setzt jeden negativen Wert auf 0)

3. Pooling

Abschliessend: Anwenden der Pooling-Matrix (z. B. MaxPooling)

CNN: Convolution + Pooling



10.4. Zusammenfassung

- MLP** — Gut für strukturierte Daten (z. B. Tabellen) 10.1
- CNN** — Gut für unstrukturierte Bilddaten 10.3
- Versteckte Schichten** — meist ReLU
- Ausgabeschicht** — Sigmoid bei binärer Klassifikation; Softmax bei Mehrklassenklassifikation 10.2
- Training** — Lernen erfolgt über Anpassung der Gewichtungen w und Bias-Terme b

11. Unsupervised Learning

Unsupervised Learning

Lernen ohne vorgegebene **Labels** bzw. **Zielwerte**. Der Algorithmus erkennt **Strukturen** nur anhand der Eingabedaten.

11.1. Clustering

11.1.1. K-Means: ähnliche Punkte gruppieren

K-Means

K-Means ist ein **unüberwachter Lernalgorithmus**, der n Datenpunkte in K **disjunkte Cluster** aufteilt, indem die **Varianz innerhalb der Cluster (Intra-Cluster-Varianz)** minimiert wird.

11.1.2. Zielfunktion

Loss Function

Sei:

- $X = \mathbf{x}_1, \dots, \mathbf{x}_n$ — mit $\mathbf{x}_i \in \mathbb{R}^d$
- μ_k — **Centroid**: Mittelpunkt der Punkte in Cluster k (kein echter Datenpunkt nötig)
- C_k — Menge der Punkte im Cluster k

Die **Verlustfunktion (Distortion bzw. Inertia)** lautet:

$$J = \sum_{k=1}^K \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \mu_k\|^2$$

Lesen: Kleine Distortion = kompakte Cluster. **Ziel:**

$$\min_{C_1, \dots, C_K, \mu_1, \dots, \mu_K} J$$

11.1.3. Lloyd's Algorithm

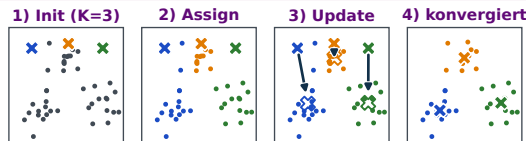
Standard K-Means

- Initialisierung**
Wähle K initiale **Centroide** (zufällig oder **k-means++**).
- Zuweisung**
Weise jeden Punkt seinem **nächstgelegenen Centroid** zu.
- Update**
Setze jeden Centroid auf den **Mittelwert** seiner zugewiesenen Punkte.
- Wiederhole**
Schritte 2–3, bis sich die **Zuweisungen nicht mehr ändern**.
- Mehrere Starts**
verschiedene Initialisierungen testen, um schlechte **lokale Optima** zu vermeiden.

K-Means terminiert immer

J fällt in jedem Schritt **monoton** und es gibt nur **endlich viele mögliche Zuordnungen**, also **terminiert** der Algorithmus garantiert. Die Initialisierung bestimmt nur, **welches lokale Optimum** erreicht wird — **nicht**, ob er terminiert.

Lloyd-Algorithmus



11.1.4. Elbow-Kriterium: sinnvolles K wählen

Um K zu wählen, berechne $J(K)$ für verschiedene Werte von K und plote:

- x-Achse** — K (Anzahl Cluster)
- y-Achse** — $J(K)$ (Within-Cluster Sum of Squares)

Der **Elbow-Punkt** ist dort, wo zusätzliche Cluster nur noch **marginalen Gewinn** bringen — die Kurve macht einen **sichtbaren Knick**.

11.1.5. K-Means mit scikit-learn

Faustregeln

- n_clusters / K** — Von dir vorgegebene Anzahl gesuchter Gruppen; mit **Fachwissen** oder **Elbow-Kriterium** wählen.
- Skalierung** — optional vorher **skalieren**, wenn Features **sehr unterschiedliche Größenordnungen** haben.

Merksatz

$n_clusters$ legt die **Anzahl Cluster** fest. Nach $\text{fit}(X)$ stehen die **Zuordnungen** in labels_ .

K-Means: Pflichten schema

```
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=3, random_state=0) # ANPASSEN: Anzahl Cluster
kmeans.fit(X)
cluster_labels = kmeans.labels_ # Cluster-IDs (keine Klassenlabels)
```

11.2. Dimensionsreduktion

11.2.1. Grundidee

Dimensionsreduktion

Dimensionsreduktionsverfahren stellen **hochdimensionale Daten** in **weniger Dimensionen** dar, wobei die wesentliche **Struktur erhalten bleibt**. Dies erleichtert **Visualisierung**, **Rauschunterdrückung** und beschleunigt **Lernverfahren**.

Feature Selection vs. Dimensionsreduktion

- Feature Selection** — wählt eine **Teilmenge der originalen Features** aus; das Ergebnis ist eine **Teilmenge**.
- Dimensionsreduktion** — bildet **neue Achsen** aus **Kombinationen der Features** (z. B. PCA); das Ergebnis sind **neue Achsen**.

11.2.2. PCA (Principal Component Analysis): wichtige Achsen

PCA ist ein **lineares Verfahren**, das neue **orthogonale Achsen (Hauptkomponenten)** bestimmt, entlang derer die **Varianz der Daten maximal** ist. Die **erste Hauptkomponente** erfasst die **grösste Varianz**, die zweite die nächst-grösste, usw. Die Daten werden anschliessend in dieses neue Koordinatensystem **projiziert**, sodass man von d auf $m \ll d$ Dimensionen reduzieren kann, während der grösste Teil der **Informationsstreuung** erhalten bleibt.

11.2.3. t-SNE: lokale Gruppen sichtbar machen

t-SNE ist ein **nichtlineares Verfahren**, das vor allem **lokale Nachbarschaften** im Datensatz erhält. Es betont **kleine Distanzen** und bildet dadurch **ähnliche Punkte nahe beieinander** ab, was oft zu **klar getrennten Clustern** in 2D oder 3D führt. Es eignet sich besonders für die **Visualisierung komplexer Strukturen**.

11.2.4. PCA und t-SNE mit scikit-learn

Faustregeln

- n_components** — 2–3 für **Visualisierung**; höher, wenn die reduzierten Daten weiterverarbeitet werden.
- PCA** — **linear**, **schnell** und auf **maximale Varianz** ausgerichtet.
- t-SNE** — **nichtlinear** und besonders für die **Visualisierung lokaler Gruppen** geeignet.

Daten vorbereiten: tabellarisch

```
from sklearn.preprocessing import StandardScaler
X = StandardScaler().fit_transform(X) # OPTIONAL bei unterschiedlichen Skalen
```

Daten vorbereiten: Bilder

```
X = images.reshape(len(images), -1) / 255.0 # flach machen & normieren
```

PCA: Pflichten schema

```
from sklearn.decomposition import PCA
model = PCA(n_components=2)
X_reduced = model.fit_transform(X)
```

t-SNE: Alternative für Visualisierung

```
from sklearn.manifold import TSNE
model = TSNE(n_components=2, random_state=0)
X_reduced = model.fit_transform(X)
```

Schlusswort

Ich wünsche allen viel Erfolg bei der Prüfung! Die ZSF soll bei Theoriefragen und beim Umsetzen von Programmieraufgaben helfen: Code-Skelette zeigen übertragbare Grundmuster. Fachliche Inspiration: Nina Burren, Georg Niggli und Eduard Fell. Fehler oder Ergänzungen gerne als Issue melden.

Links & Ressourcen:

Aktuelle Version & Hliddal Template:

garcon-studios.ch/zsf

HINWEIS ZUM DRUCKEN

Beim Drucken **2 Seiten pro Blattseite** wählen — so werden die 16 PDF-Seiten zu den erlaubten 8 Druckseiten (entspricht 4 A4-Blättern). Je zwei PDF-Seiten bilden eine Druckseite (1a+1b bis 8a+8b).